

The Alberta Buck - Ethereum Implementation (DRAFT v0.94)

Perry Kundert

2026-05-24 12:00:00



A concrete Ethereum implementation of the Alberta Buck Architecture, in six contracts:

1. **BUCK_CREDIT** (ERC-721) – one NFT per insured asset, with deterministic depreciation and piecemeal client activation.
2. **IdentityRegistry** – per-address Pointcheval-Sanders credentials and ElGamal-encrypted identity points, verified on-chain via the BN254 precompiles. Trust anchor for every BUCK transfer.
3. **BUCK** (ERC-20) – identity-bound, single-slot per-account state with *signed* raw balances, continuous demurrage on positive holdings, and NFT-backed credit headroom that lets the raw balance go negative up to `creditLimit(a) = totalCurrentValue(a) * currentBuckK / 1e18`. `mint(N)` activates NFT-backed coverage and pays the insurance pool principal out of the holder's credit (no BUCK is delivered to the holder's raw balance); the holder spends the new headroom by transferring out, driving the signed raw negative. `balanceOf` reports `held + unused credit` on non-Carrying accounts, so the ERC-20 view answers "*what can I spend right now*".
4. **BuckBasket** – USD-free basket registry and direct-mint orchestrator. Custodies a single full-range Uniswap-V3 LP position per TOKEN/BUCK pool, mints BUCK against deposited basket tokens at the current pool spot price, and on redemption withdraws proportionally from overweight pools, hands the depositor the TOKEN side(s) and recycles the BUCK side "sell high / buy low" into the most underweight pool as treasury LP. Publishes `basketValueInBuck() = sum( basketAmount_i * pool_price_in_BUCK_i )` as the direct-embodiment process variable for BUCK_K.

5. **BUCK_K** – the value-stabilization factor for credit-limit computation. Three implementations ship today: `BuckKControllerStatic` (governance-set constant), `BuckKControllerDirect` (USD-free PID; setpoint = 1.0 BUCK by definition, process variable = `BuckBasket.basketValueInBuck()`), and `BuckKController` (PID against an external four-pool USDT/USDC basket of Uniswap-V3 TWAPs, with a BUCK/USDT TWAP as the BUCK price). The PID mechanics – priming, setpoint-shift (dS) compensation, long-gap dTMax clamp, `fundingFactor()` counter-cyclical insurance view – live in a shared `BuckKControllerBase` following the `ezpwd::pid` pattern; `Buck.mint` and `Buck.burn` drive `compute()`, and `BuckBasket` holds the privileged hook to `reprime()` after constituent additions.
6. **Notes** – privacy-preserving denomination on top of BUCK. Mint and spend via Groth16 SNARKs over a Tornado-style incremental Poseidon Merkle tree.

This is the first implementation, prioritizing clarity over breadth of standards. Subsequent documents will examine ERC-1155, ERC-3643, and ERC-4626 alternatives for individual layers.

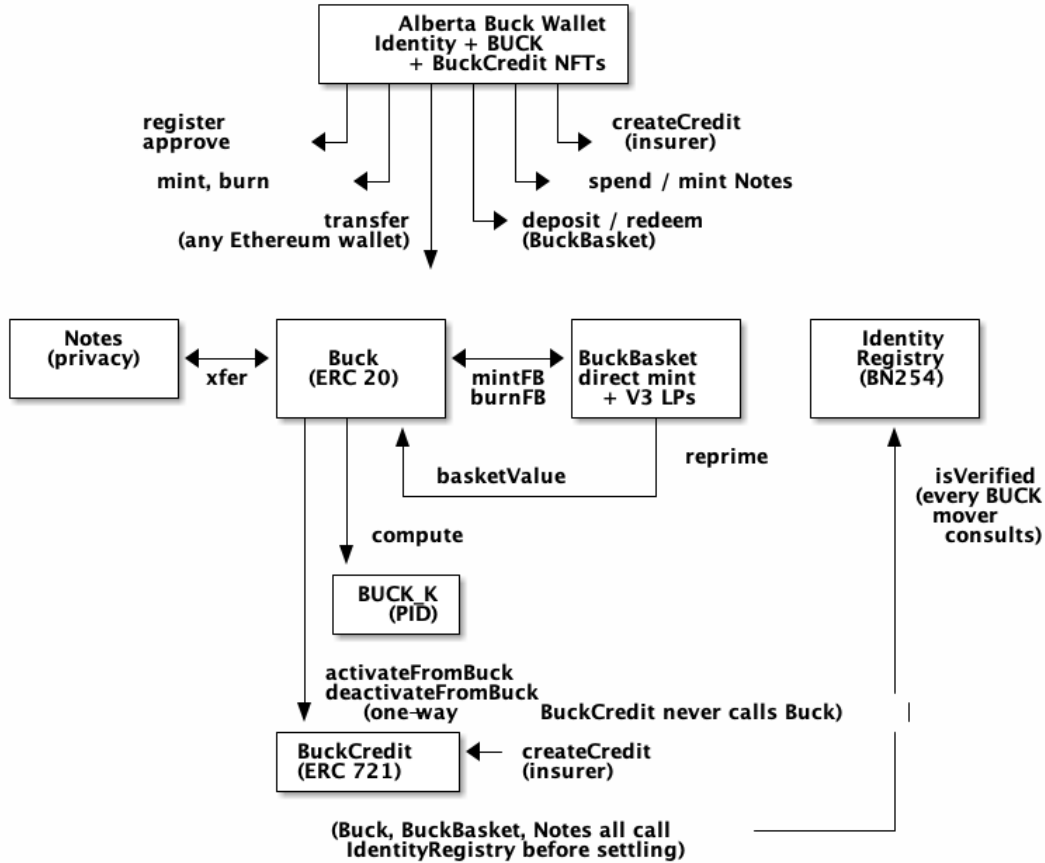
Contents

1	Contract Architecture	5
2	ERC Standard Choice for BUCK_CREDIT	7
3	BUCK_CREDIT: ERC-721 Insured Asset NFT	7
3.1	Data Model	7
3.2	Depreciation Models	7
3.3	Activation = Mint, Deactivation = Burn	8
3.4	Buck Wiring	9
3.5	Insurer Updates	9
4	IdentityRegistry: On-Chain Trust Anchor	10
4.1	Cryptographic Surface	10
4.2	Solidity Surface	11
4.3	Public-Identity Contracts	12
4.4	Issuer Trust	12
5	BUCK: ERC-20 Token	12
5.1	Storage Layout	13
5.2	Signed Balances and <code>balanceOf</code>	13
5.2.1	The Credit Limit Is Read Live	14
5.3	<code>mint(N)</code> and <code>burn(N)</code>	14
5.3.1	The Per-Credit Inversion	14
5.3.2	Releasing: the Deposit Comes Back	15
5.3.3	Underwater Is a State, Not a Failure	17
5.4	Identity-Bound <code>approve()</code>	18
5.5	<code>transfer()</code> and <code>transferFrom()</code>	19
5.5.1	Reentrancy Guard	19
5.6	Demurrage and Jubilee	20
5.6.1	Delegated Demurrage: a Designated Fee Payer	21
5.7	Integrating with BUCK	22
6	BuckBasket: Direct-Mint Orchestrator + USD-Free Value Anchor	22
6.1	Two Kinds of BUCK	22
6.2	Constituents and Weights	23
6.3	Process Variable: <code>basketValueInBuck()</code>	23
6.4	Direct-Mint: <code>depositToken</code>	24
6.5	Redemption: "Sell High, Recycle to Buy Low"	24
6.6	Invariant	25
6.7	Known Gaps	25
6.8	Dynamic Issuance: Why BuckBasket Needs BUCK_K	25
7	BUCK_K: Value Stabilization Controller	27
7.1	Static Variant	27
7.2	Direct Variant (USD-Free)	27
7.3	External-Basket Variant	28
7.3.1	External Basket and BUCK Price Oracles	28

7.3.2	Priming and dS Compensation (shared with the Direct Variant)	29
7.3.3	dT, dTMax, and Pokability (shared with the Direct Variant)	29
7.3.4	Future: Amortizing PID Work via a State Machine	30
7.3.5	Tuning (shared with the Direct Variant)	30
8	Notes: Privacy-Preserving Denomination	30
8.1	Mint Circuit (Batch, In-Circuit Merkle Insertion)	30
8.2	Spend Circuits (B-flavor, A-flavor V2)	31
8.3	Verifier Interfaces	31
8.4	Notes Contract	32
9	End-to-End Walkthrough	33
9.1	Step 0: Identity Registration (One-Time)	33
9.2	Step 1: Mint	33
9.3	Step 2: Approve Bob (Identity Handshake)	34
9.4	Step 3: Bob Pulls the Transfer	34
9.5	Step 4: Alice Denominates as Notes	34
9.6	Step 5: Spend a Note	35
10	Gas Cost Snapshot	35
11	Security Considerations	36
11.1	Oracle Manipulation	36
11.2	Unvetted Insurers – the open hole	36
11.3	Recipient Opt-In	37
11.4	Insurer Trust	37
11.5	Reentrancy	38
11.6	Pool Solvency	38
12	Future Directions	38
13	Status	39

1 Contract Architecture

The six contracts compose into a single transfer pipeline gated on identity. **IdentityRegistry** is the trust anchor: **mint**, **approve** and **transfer** all consult it before mutating state. Every account *and* contract that touches BUCK carries an identity binding – EOAs via **register**, contracts via **bindContract** – so bilateral identity-checked transfers always have a verifiable counterparty on both sides.



A Wallet (EOA) is the only initiator. It registers once with **IdentityRegistry** and then mints, burns, transfers, deposits or spends via the contract entry points in the second row. Every contract that moves BUCK – Buck, BuckBasket, Notes – consults **IdentityRegistry.isVerified** on both counterparties before settling. **BuckCredit (ERC-721)** is mutated only by insurers (**createCredit**, **updateCredit**) and by Buck itself (**activateFromBuck** / **deactivateFromBuck** during mint and burn); the dependency is one-way – **BuckCredit** never calls back into **Buck**. **BUCK_K** is private infrastructure: **Buck** calls **compute()** on every mint/burn, and **BuckBasket** is the privileged caller of **reprime()** and the source of **basketValueInBuck()** that the **Direct** variant integrates against. External dependencies (not shown): Uniswap V3 (LPs custodied by **BuckBasket**; TWAPs read by the **External** variant of **BUCK_K**); the BN254 EVM precompiles (pairings for **IdentityRegistry**, modular arithmetic for the **Mint/SpendVerifier** contracts); circom / snarkjs for off-chain **Notes** proof generation.

The flow is:

1. A **client** names an insurer it will accept credits from (`setCreditIssuer`), and that **insurer** mints a `BUCK_CREDIT` NFT for the client's asset, setting face value, depreciation schedule, and `premiumRate` (annual basis points of activated coverage). A credit only lands where its recipient asked for it – see Unvetted Insurers for what that does and, more importantly, does not protect against.
2. There is no separate activation step, and no public `BuckCredit.activate`. Coverage is activated only inside `Buck.mint`, atomically with the insurance principal that funds it – see `Activation = Mint`, `Deactivation = Burn`. While coverage is live the credit cannot change hands.
3. Before touching `BUCK`, the client **registers** with `IdentityRegistry`, posting a blinded presentation (`A`, `B`) of its PS credential, an ElGamal-encrypted identity point and a proof tying the two to the account key. The registry verifies the proof via BN254 pairings (443,892 gas measured, paid once per address).
4. `BUCK.mint(amount[, tokenIds])` draws coverage cheapest-first across the client's NFTs, activates it, and debits the mutual-insurance pool principal – 10x the annual premium on the coverage – from the client's *signed* raw balance into `insurancePool1`. No `BUCK` is delivered to the client's raw balance; what the client gains is spendable headroom. The arithmetic is in `mint(N)` and `burn(N)`.
5. To send `BUCK`, the sender **approves** with a Chaum-Pedersen receipt that re-encrypts the sender's identity under the spender's public key. `transfer` / `transferFrom` then bilateral-identity-check both sides and emit `BuckTransferReceipt` carrying only ciphertext hashes.
6. To denominate as private notes, the client calls `Notes.mint` with a Groth16 proof binding the deposited `totalFace` to a batch of Poseidon commitments. Holders later call `Notes.spend` or `Notes.spendACP` with a fresh nullifier and a SNARK proving inclusion under a recent root.
7. `BuckBasket` is the **direct-mint orchestrator** and the value anchor. Anyone holding a basket constituent (PAXG, cbBTC, XAUT, WBTC, etc., as wired by governance via `addBasketToken`) can call `BuckBasket.depositToken` to LP it into the corresponding `TOKEN/BUCK` Uniswap-V3 pool; the basket mints exactly `tokenAmount * spotPrice` worth of fresh `BUCK` against the deposit (via the privileged `Buck.mintFromBasket` hook), pairs the two sides as full-range V3 liquidity, and hands the depositor an ERC-721 receipt (`BuckBasketReceipt`). Depositing already-minted `BUCK` is symmetric: the basket swaps it into the most underweight constituent's `TOKEN` and LPs there. On `BuckBasket.redeem`, the value claim is allocated across overweight pools proportional to each pool's positive value-weight error; the basket withdraws `L` from each, transfers the entire `TOKEN` side to the depositor, burns the principal `BUCK` from the `BUCK` side, and routes the remaining "profit" `BUCK` into the most underweight pool's `TOKEN` as treasury LP – the "sell high on the way out, recycle to buy low" leg fires in the same transaction.
8. `BUCK_K` ships in three forms today. The governance-set static variant is the simpler choice for early deployments where no on-chain price reference exists. `BuckKControllerDirect` is the **USD-free** embodiment: process variable = `BuckBasket.basketValueInBuck()`, setpoint = 1.0 `BUCK` by definition, error = `1.0 - basketValue`. The legacy `BuckKController` reads its process variable (`BUCK` price) from a `BUCK/USDT` TWAP and its setpoint (basket cost) from a four-pool `USDT/USDC` basket – `XAUT/USDT`, `PAXG/USDC`, `cbBTC/USDC`,

WBTC/USDT, 25 % each – chosen for resilience against any single basis-currency or asset-token freeze. `Buck.mint` and `Buck.burn` call `BuckK.compute()` regardless of variant, so user activity drives and amortizes the PID; long quiet stretches are bounded by `dtMax`, and `BuckBasket` is the privileged caller of `reprime()` after `addBasketToken` so dilution discontinuities don't fire a one-cycle P/I spike.

2 ERC Standard Choice for BUCK_CREDIT

ERC-721 was the obvious starting point and the choice that ships. Each credit is genuinely unique (specific asset, depreciation curve, insured value, premium schedule); `ERC721Enumerable` gives `BUCK.mint` the `tokenOfOwnerByIndex` it needs to walk a holder's NFTs. ERC-3643 (T-REX) is a plausible future move if regulatory KYC becomes a hard requirement; ERC-1155 helps only if standardized credit *classes* emerge with shared depreciation parameters; ERC-4626 fits the `InsurancePool` but not individual credits. None of those are revisited below – the rest of the document specifies the shipped ERC-721 path.

3 BUCK_CREDIT: ERC-721 Insured Asset NFT

Each `BUCK_CREDIT` is an `ERC721Enumerable` token whose per-token `CreditParams` holds the insurer's current offer and the client's activation state. Three roles touch it: the **insurer** mints and updates, the **client** activates, `BUCK` reads the depreciated activated value to size mint allocations. All monetary fields are in 6-decimal `BUCK`-equivalent units (USDC-compatible).

3.1 Data Model

Fields group by mutability:

Issuance is opt-in per (client, insurer) pair: `createCredit` reverts unless the client has called `setCreditIssuer(insurer, true)`. Anyone may still *be* an insurer – that is the open question in Unvetted Insurers – but nobody can put a credit in your hands uninvited.

- **Immutable at mint:** `insurer` (the address allowed to update this token), `assetClass`, `createdAt`.
- **Insurer-mutable** via `updateCredit`: `faceValue`, `depreciationFloor`, `depType`, `depRate`, `depStartAt`, `premiumRate`, `lastUpdated`. `faceValue` may not be reappraised below `activatedValue` – see Insurer Updates.
- **Buck-mutable** via `activateFromBuck` / `deactivateFromBuck`: `activatedValue` (capped at `faceValue`), `lastActivatedAt`. The client never writes these directly.

`DepreciationType` is one of `NONE` (land, gold), `LINEAR` (constant annual reduction), or `DECLINING_BALANCE` (continuous-compounded percentage). Three events – `CreditCreated`, `CreditUpdated`, `CreditActivated` – record every state change.

3.2 Depreciation Models

`currentValue(tokenId)` is a pure view that branches on `depType`:

- `NONE`: returns the activated face value, unchanged.

- **LINEAR**: loss accrues at `depRate` bp/year on the depreciable portion (`faceValue - floor`), clamped at `depreciationFloor`.
- **DECLINING_BALANCE**: discrete whole-year compounding – the factor $(BP - \text{depRate})/BP$ is applied once per complete year via a loop capped at 100 years, then a linear interpolation bridges the trailing partial year. No transcendental approximations.

In every case the activated portion depreciates proportionally: `currentValue = depreciatedFace * activatedValue / faceValue`.

Asset	Type	Rate	Floor	30yr value (of \$100K)
Land	NONE	–	–	\$100,000
House (bldg)	LINEAR	200bp	20%	\$52,000
Vehicle	DECLINING_BALANCE	1500bp	5%	\$5,725
Farm equip	DECLINING_BALANCE	1000bp	10%	\$13,815
Gold in vault	NONE	–	–	\$100,000

A residential property would typically have two `BUCK_CREDITS`: one for the land (NONE) and one for the structure (LINEAR or DECLINING_BALANCE with a salvage floor).

3.3 Activation = Mint, Deactivation = Burn

There is no public `BuckCredit.activate()` function. Activation – expanding `activatedValue` and thus growing the holder’s `creditLimit` – happens *only* atomically as part of `Buck.mint(N, [tids])`, and deactivation happens *only* atomically as part of `Buck.burn(N, [tids])`.

This is a deliberate consequence of the mutual-insurance pool model. Drawing `take` units of coverage on an NFT generates `annual_premium = take * premiumRate / BP` per year. Instead of charging that premium on a continuous schedule, the contract requires an *upfront* `poolPrincipal = annual_premium * POOL_ROI_INV` deposit (where `POOL_ROI_INV = 10` encodes the insurer’s assumed 10 % annual ROI on pool capital). At parity, the pool’s investment yield on `poolPrincipal` exactly funds the annual premium on `take` in perpetuity – so *a policy is a one-time purchase*, not a recurring expense.

Therefore the surface `BuckCredit` exposes is restricted to (a) insurer-side creation / re-appraisal (`createCredit`, `updateCredit`), (b) standard ERC-721 ownership operations (`transferFrom`, `balanceOf`, `ownerOf`, ...), and (c) two Buck-only entry points (`activateFromBuck`, `deactivateFromBuck`) which `Buck._allocateMint` / `_allocateBurn` call as part of their atomic sequences. Test fixtures that need to set up a holder with non-zero `creditLimit` without exercising the `mint()` purchase path use the `BuckCreditHarness` subclass’s `forceActivate` – explicitly not present on production `BuckCredit`.

`Buck.mint` and `Buck.burn` are the only callers. Their per-NFT arithmetic, and the post-burn solvency rule that goes with it, are in `mint(N)` and `burn(N)`.

A credit that is backing BUCK cannot change hands. `BuckCredit._update` – the ERC-721 hook every mint, transfer and burn passes through – refuses to move a token whose `activatedValue` is non-zero, with `BuckCredit: credit in use`. Minting is exempt (`from = address(0)=`), so `createCredit` still works, and a burn that takes coverage back to zero frees the credit to be sold again.

The reason is that the two halves of a credit position live in different places. Buck derives the holder’s `creditLimit` from the activated value of the credits they *own*, while the BUCK drawn against that limit stays as a negative raw balance on the account that *drew* it. Let the token move

and the two separate: the seller keeps an obligation with nothing behind it, and the buyer receives headroom against coverage that has already been spent. The same activated value would back BUCK twice.

`activatedValue` is the predicate rather than `Buck's mintsBacked` deliberately: it is local, so `BuckCredit` enforces the property without consulting `Buck` at all, and the guarantee does not depend on another contract being correct or even reachable. The two agree by construction – activation happens only inside `Buck._allocateMint`, and `updateCredit` may not reappraise below activated coverage.

`totalCurrentValue(account)` walks `tokenOfOwnerByIndex` and sums `currentValue(tokenId)` – the single call `Buck.creditLimit` makes to size the holder's headroom (then multiplied by `currentBuckK` and divided by `1e18`). `creditInfo(tokenId)` returns (`faceValue`, `activatedValue`, `premiumRate`) one NFT at a time; `batchCreditInfo(uint256[] tokenIds)` returns a packed `CreditSlice[]` = (`owner`, `faceValue`, `activatedValue`, `premiumRate`) for the entire list in one external call, used by `Buck._allocateMint` / `_allocateBurn` to amortize per-NFT cross-contract dispatch overhead.

3.4 Buck Wiring

The dependency is one-way: `Buck` reads `BuckCredit` to size credit, and `BuckCredit` never calls `Buck`. `BuckCredit.setBuck(address)` is a one-shot post-deployment authorisation record – it names the single contract allowed to call `activateFromBuck` / `deactivateFromBuck` – and nothing more.

Earlier drafts had `BuckCredit` notify `Buck` on every NFT mutation, so that a per-block credit-limit cache in `Buck` could invalidate. Both the cache and the notification are gone; see *The Credit Limit Is Read Live* for why the cache could not have been made correct. Removing them left the call graph acyclic, which is worth more than the cache was: there is no reentrant path from `BuckCredit` back into `Buck`, so `Buck's` reentrancy guard needs no exemption for one.

The two-contract split, rather than collapsing into a single Diamond, is forced by the ERC-20 / ERC-721 standards: `balanceOf(address)` and `transferFrom(address,address,uint256)` share function selectors with incompatible semantics across the two standards, and the `Transfer` event has different indexed-argument counts. A Diamond can dispatch one selector per signature; whichever loses stops being standards-compliant and silently breaks wallets, indexers, marketplaces, and routers. See *Future Directions* for the modularization paths that *are* viable (intra-ERC-20 facets in `Buck`, independent UUPS for `BuckCredit`).

3.5 Insurer Updates

`updateCredit(tokenId, newFaceValue, newDepreciationFloor, newDepType, newDepRate, newDepStartAt, newPremiumRate)` is gated on `msg.sender == credits[tokenId].insurer.assetClass` and `insurer` never change.

An insurer may reappraise down to the coverage the holder has already bought, and no further: `newFaceValue >= activatedValue`, else `BuckCredit: face below activated coverage`. Activated coverage is a completed purchase – its pool principal was paid up front and funds its premium in perpetuity – so writing it down revokes a policy rather than revaluing an asset. An asset that has genuinely lost value is what a **claim** is for; ordinary decline is what the depreciation schedule is for, and that still moves the holder's credit limit continuously without touching the coverage itself. Every other lever stays: `faceValue` down to the activated line, the schedule, the floor, the premium rate.

The rule is also load-bearing for solvency. `activatedValue` lives in `BuckCredit` and `mintsBacked` lives in `Buck`; the burn is sized from the second and checked against the first. An earlier draft let a reappraisal clamp one without the other, and a holder whose credit was written down could then never unwind the position – burnable only down to the clamped line, with the remainder stuck and the credit permanently disqualified from backing BUCK again. Refusing the clamp keeps the two equal by construction, which is also what makes the transfer lock in `Activation = Mint` the right predicate.

A reappraisal, or ordinary depreciation, can still leave a holder’s outstanding BUCK above their credit limit. That is a supported state, not a default: see *Underwater Is a State, Not a Failure*.

4 IdentityRegistry: On-Chain Trust Anchor

The `IdentityRegistry` is the binding between Ethereum addresses and the privacy-preserving identity machinery described in the companion documents *Identity with Anonymity*, *Identity Example – Data Flow*, and *Cryptographic Proofs*. Every BUCK `mint`, `approve` and `transfer` consults this registry; an unregistered address cannot hold or move BUCKs.

The registry stores per address: `pk[a]` (G₁ identity public key), `E_addr[a]` (ElGamal ciphertext of the identity point M), `isVerified` (true after PS+NIZK pass on-chain or after `bindContract`), `isCarrying` (controls Buck’s transfer dispatch), and `isPublicIdentity` (true only for contracts bound with the public-disclosure flag). Trusted-issuer PS public keys live in `trustedIssuers[issuerAddr]`, populated by governance. EOAs always self-register; contracts always `bindContract`.

4.1 Cryptographic Surface

Four on-chain verifications, all reducible to BN254 precompile calls:

1. **PS credential presentation at registration:** the holder publishes $(A, B) = (a * \text{sigma}_1, a * \text{sigma}_2 + b * Y1)$ for fresh a, b , with $Y1 = y * G$ the issuer’s G₁ key component, and the registry checks the masked relation $e(B, g_2) = e(A, X) * e(m * A + b * G, Y)$ through the proof in step 2 – one three-pair pairing product. The published pair is uniform whatever the identity, so no holder of m (the issuer included) can test it; a rerandomized signature, which an earlier design published, would be a valid signature on m and testable by anyone holding it.
2. **Schnorr-family NIZK** binding the presentation to the ElGamal ciphertext and the account key. Four response scalars over one credential commitment $C1 = m * A + b * G$ (so no field exposes $m * A$ alone), Fiat-Shamir bound to the action/version domain (`AlbertaBuck/FiatShamir/IdentityRegistry/Register/v3`), $((A, B), E_{\text{new}}, pk_{\text{new}}, \text{registrant_addr})$, chain ID, and registry deployment. Total registration 443,892 gas measured (472,630 before the presentation), paid once per address; `trustIssuer` additionally checks $e(Y1, g_2) = e(G, Y)$ once per issuer.
3. **Chaum-Pedersen NIZK at approve:** proves the recipient’s identity ciphertext re-encrypts the same M registered in `E_alice` and that the sender holds the registered account key. Fiat-Shamir binds the registry deployment, `msg.sender`, `spender`, and `chainid`. ~29K gas per counterparty pair.
4. **Chaum-Pedersen DLEQ at A-flavor spend** (`verifySpendCP`): proves the spender holds a single `sk_dep` that opens both the note’s `E_n` and the registered `E_reg`. ~36K gas via EIP-196.

4.2 Solidity Surface

src/IdentityRegistry.sol uses the BN254 helper for all curve math:

```
struct G1Point { uint256 X; uint256 Y; }
struct G2Point { uint256[2] X; uint256[2] Y; }

struct ElGamalCT { G1Point R; G1Point C; } // (R, C) = (r*G, r*pk + M)
struct PSPresentation { G1Point A; G1Point B; } // (a*sigma_1, a*sigma_2 + b*Y1)
struct PS PubKey { G2Point X; G2Point Y; G1Point Y1; } // Y1 = y*G, checked at trustIssuer

struct RegistrationProof { uint256 e, s_m, s_b, s_r, s_sk; G1Point C1, T_C, T_R, T_key; }
struct CPProof { uint256 e, s1, s2; G1Point T1, T2, T3; }
struct SpendCPProof { uint256 e, s; G1Point T1, T2; }
struct ContractBindingProof { uint256 e, s; G1Point T; }

function register(address issuer, G1Point pk, ElGamalCT E,
                  PSPresentation pres, RegistrationProof proof);
function authorizeContractBinding(address binder, G1Point pk, ElGamalCT E,
                                  bool isPublicIdentity_, bool isCarrying_);
function bindContract(address target, G1Point pk, ElGamalCT E,
                      bool isPublicIdentity_, bool isCarrying_);
function bindContract(address target, address issuer, G1Point pk, ElGamalCT E,
                      PSPresentation pres, RegistrationProof credential,
                      ContractBindingProof holderAuthorization,
                      bool isPublicIdentity_, bool isCarrying_);
function bindContractFromAdapter(address target, address operator,
                                  bool isPublicIdentity_, bool isCarrying_);
function setBindingAdapter(address adapter, bool approved); // governance
function trustIssuer(address issuer, PS PubKey ipk); // governance
function verifyApprove(address sender, address spender,
                       ElGamalCT E_bob, CPProof pi) view returns (bool);
function verifySpendCP(address spender, address recipient,
                       ElGamalCT E_n, SpendCPProof cpProof) view returns (bool);
function markApproved(address spender); // Buck-only
function setBuck(address buck); // governance, one-shot
```

A few non-obvious points:

- **verifyApprove** reads `E_alice` from registry storage so the caller cannot substitute a fake credential. Its transcript binds the verifying registry address, so a proof prepared for a different registry deployment is rejected. Here "registry" means `address(this)` while the proof is verified: for a Diamond facet that is the stable Diamond proxy, not the replaceable facet implementation. Replacing a facet therefore does not invalidate proofs. Moving the registry to a different proxy requires unsubmitted registration/approve proofs to be regenerated (and registry state to be migrated), but does not alter already minted Notes or their receipts; those artifacts do not contain either proof.
- **bindContract** requires both certification and target control. The compact overload copies a registered caller's exact stored identity after the target authorizes the full binder/identity/

policy tuple. The credential overload verifies PS certification plus holder authorization; target authorization remains mandatory on that path. Contracts with known provenance use a governance-approved, contract-specific adapter. The adapter proves canonical provenance and authority, while the registry copies the named operator’s stored identity rather than accepting adapter-supplied keys. The first authorized binding is immutable.

- **markApproved** (called by Buck inside **approve**) freezes the spender’s **isCarrying** flag the first time anyone successfully approves them, so the carrying flavor a sender consents to cannot be retroactively flipped.
- Revocation does not un-register existing accounts; they were verified once and are never re-verified.

4.3 Public-Identity Contracts

DeFi infrastructure – AMM pools, vaults, oracles, the Notes pool – can’t generate Chaum-Pedersen proofs at **approve** time because the contract has no private key. Such contracts authorize and bind a certified operator identity, or use an approved adapter with audited provenance and authority semantics, with **isPublicIdentity_=true**, **isCarrying=true**. Counterparties of a Public-Identity contract still call the identity-bound **approve** (the always-CP rule applies regardless of spender flavor), but the receipt-fragment check on the contract side is satisfied by the deterministic **_identityHash** rather than a per-counterparty CP receipt. Public-Identity contracts forfeit content privacy: their identity M is openable on subpoena via the operator’s off-chain attestation. Encrypted-Identity contract binding (**isPublicIdentity_=false**) is supported by the surface but not yet used in production.

4.4 Issuer Trust

trustedIssuers is the only governance-controlled trust anchor. Adding an issuer admits a new origin of registrable identities; removing one stops new registrations under that issuer. Existing registrations are unaffected – they were verified at registration and never re-verified. Epoch-based credential renewal, key revocation, and issuer rotation are described in Identity: Epoch-Based Credential Renewal.

5 BUCK: ERC-20 Token

BUCK is an identity-bound ERC-20 with NFT-backed credit headroom, mutual-insurance-pool minting, and continuous demurrage. All transfer-path state for an account fits in one storage slot, so a transfer is one SSTORE per side. **mint(N)** opens enough NFT-backed insured-asset coverage to deliver N units of *spendable headroom* to the holder (no BUCK is delivered to raw balance) and simultaneously transfers a *pool principal* from the holder’s signed raw to **insurancePool**, sized so that at the insurer’s assumed annual ROI the principal’s investment yield exactly covers the annual premium on the activated coverage. The holder spends the headroom by transferring out, which drives their signed raw negative; **burn(N)** unwinds coverage most-expensive-first and refunds principal back to the holder, lifting their signed raw toward zero. Every transfer routes through the **IdentityRegistry** and emits **BuckTransferReceipt(from, to, amount, fromHash, toHash)** – ciphertext hashes, no plaintext identity on chain.

5.1 Storage Layout

The transfer-path state for an account is one slot:

```
struct AccountState {
    BuckQty    balance;           // int80 underlying; signed. Range = [-6.04e23, +6.04e23] raw
                                   // = [-6.04e17, +6.04e17] BUCK at 6 decimals. Negative ==
                                   // NFT-backed used credit; positive == held BUCK.
    BuckSeconds buckSeconds;      // uint120 underlying; demurrage IOU integral on positive holding
    uint40      timestamp;        // last crystallisation (safe past year 36800)
    uint16      flags;            // bit 0: demurrage routed to demurragePayer[a].
                                   // bits 1-15 reserved.
}
mapping(address => AccountState) internal _state;
```

`BuckQty` is a user-defined value type wrapping `int80`; constructors `toBuckQty(uint256)` (bounds-checks against the positive cap) and `toBuckQtySigned(int256)` (bounds-checks both endpoints) live in `BuckTypes.sol` so a future width change propagates to `Buck` and `BuckCredit` in lockstep. `BuckCredit` always reads via `asUInt()` (its face, floor, and activated values are non-negative by construction); `Buck` reads via `asInt()` in signed contexts and `asUInt()` only after verifying non-negativity.

A bit in `flags` is the cheapest state this contract has: the word rides in the slot the transfer path already loads and stores, so testing one is free. That is exactly why it is spent only on hot-path dispatch. Cold-path bookkeeping – the count of accounts a demurrage payer carries – lives in its own mapping instead.

Outside the hot path: allowances, per-credit outstanding cover and the deposit held against it (`mintsBacked[tokenId]`, `mintsPrincipal[tokenId]`), approve-time receipt fragments, and the delegated-demurrage maps. **Nothing derived is stored.** An account's credit limit, and therefore its `balanceOf`, is recomputed from live `BuckCredit` state on every read; there is no cached copy of it anywhere. `mintsPrincipal` is not an exception – it is a record of what was actually paid, which no amount of present-day state can reconstruct once the appraisal or the rate has moved.

BUCK implements `IERC20` and `IERC20Metadata` directly – no OpenZeppelin `ERC20` inheritance, so the slot layout is explicit and the transfer paths are linear (no `_update` override re-entry). Decimals are 6, matching USDC.

5.2 Signed Balances and `balanceOf`

`signedRawBalanceOf(a)` returns the raw `int80` directly; `rawBalanceOf(a)` clamps negatives to zero for legacy ERC-20-style readers; `signedBalanceOf(a)` subtracts demurrage from positive raw (returns raw directly for negative raw, since used credit accrues no demurrage). `balanceOf(a)` – the ERC-20 view – answers *"what alice can spend right now"*:

```
held(a)          = max(0, signedRaw(a) - feeOwing(a))           // Non-Carrying
used(a)          = max(0, -signedRaw(a))
unusedCredit(a)  = max(0, creditLimit(a) - used(a))
balanceOf(a)     = held(a) + unusedCredit(a)                   // Non-Carrying
balanceOf(a)     = max(0, signedRaw(a))                         // Carrying
```

Carrying accounts (AMM pools, Notes pool, Jubilee) hold no `BuckCredit` NFTs, so `creditLimit` == 0 and the signed-balance machinery degenerates to the prior unsigned model. `feeOwing` and

`balanceOfFees` clamp to zero on negative raw – demurrage is a per-second property of **held** BUCK, not used credit.

5.2.1 The Credit Limit Is Read Live

`creditLimit(a) = totalCurrentValue(a) * currentBuckK / 1e18` is computed from scratch on every read. This is the point of `BUCK_CREDIT`, not an oversight: a holder’s spendable balance is meant to move when they acquire a credit, when an insurer reappraises one, when an asset depreciates on its schedule, and when `BUCK_K` moves under the whole system. A `balanceOf` that did not move with those would be reporting the wrong number.

An earlier draft cached the product per block, invalidated by an NFT-mutation hook from `BuckCredit`. That could not have been made correct. `BUCK_K` moves inside any `mint` or `burn` that advances the PID – from any account, with nothing to announce it – so two transactions in one block were enough to read a stale limit; and no hook can cover depreciation at all, which is continuous. The cache never populated in practice, and it was removed along with the hook rather than repaired.

The cost lands where it should:

<code>creditLimit(a)</code> for ...	gas
an account with no credits	6,327
an account with 10 credits, <code>NONE</code> schedule	27,603
one credit, 40 years into <code>DECLINING_BALANCE</code>	11,925

An account holding ten insured assets pays for ten insured assets on every outbound transfer. An account with a plain positive balance pays one external call that returns zero, and a Carrying account never reaches the credit path at all. That distribution is deliberate: drawing on insured collateral is a more expensive operation than spending cash, and it should read that way on the gas meter.

5.3 mint(N) and burn(N)

`mint(uint256 amount)` and `mint(uint256 amount, uint256[] calldata tokenIds)` are the two overloads. The no-arg form runs an internal cheapest-first selector (insertion sort over the caller’s NFT list – typically a handful) to minimise premium cost; the second form takes a caller-supplied order an off-chain optimizer pre-computes against effective rate, depreciation, or coverage midpoint.

`burn(uint256 amount)` mirrors the overload pair, but the default selector walks the holder’s NFTs **most-expensive-first** – the dearest insurance is released first, returning the largest pool principal per BUCK burned and freeing expensive coverage capacity for re-use. The asymmetry is not arbitrageable: per-NFT inversion is symmetric (same `denom` on both sides), so a mint-burn round-trip on the same NFT restores both its `mintsBacked` and `activatedValue` exactly.

5.3.1 The Per-Credit Inversion

Two quantities have to be kept apart, and the whole inversion turns on the difference.

`activatedValue` is denominated in **face** units – a slice of the asset *as appraised at issue*. What is actually insured today is that slice scaled by `rho = depreciatedFace / faceValue`. That is proportional coinsurance, and it is the right shape for a depreciating asset: the holder’s share of the asset stays constant while its worth tracks the asset. `creditLimit` reads the scaled figure, so the allocator inverts in **present insured value** and grosses back up into face units at the end.

Each credit carries an annual `premiumRate` in basis points, and **the premium is charged on what is actually insured** – the present value, not the face slice. You pay for the cover you have, not for the cover the asset used to be worth. So, given a target net headroom `amount` and rate `r`:

```
denom      = BP - r * POOL_ROI_INV      (must be > 0; max effective rate ~1000bp)
V          = ceil(amount * BP / denom)  present insured value that settles 'amount'
principal  = V - amount                = V * r * POOL_ROI_INV / BP
take       = ceil(V * faceValue / depreciatedFace)  face units that carry V
```

At `POOL_ROI_INV = 10` the pool deposit is ten times the annual premium on `V`, so at the insurer's assumed 10 % ROI its yield funds that premium in perpetuity; the insurer keeps any return above 10 % as profit. When `rho = 1` the last line is the identity and this reduces exactly to the undepreciated case.

Charging the premium on present value is not merely fairer, it is the only formulation that does not fall over. Charge on the face slice instead and the inversion becomes `take = amount * BP / (rho * BP - effRate)`, which is unsatisfiable once `rho * BP <= effRate`: a 200bp credit would become unmintable at *any* price below 20 % of face, because depreciation had eaten the premium margin. With the premium on `V`, `denom` is independent of `rho`, every credit with a non-zero appraisal still works, and the cost per BUCK delivered stays `amount * effRate / denom` however old the asset is. That last property is why cheapest-first by `premiumRate` remains the correct selector: depreciation changes how much **capacity** a draw consumes, never how much it **costs**.

The allocator (`_allocateMint`) walks `BuckCredit.batchCreditInfo(tokenIds)` once, then iterates in memory. Per credit the available present value is `capV = (faceValue - mintsBacked) * rho` and the net it can settle is `capV * denom / BP`; the credit is either fully consumed or partially consumed via the closed form above to land exactly on `remaining`. A credit whose appraisal has reached zero insures nothing and is skipped. For each credit touched, `_allocateMint` calls `BuckCredit.activateFromBuck(tid, holder, take_i)` to grow `activatedValue` and `mintsBacked` in lockstep – the activation and the principal payment are the same atomic operation.

5.3.2 Releasing: the Deposit Comes Back

The release side is deliberately **not** the mirror of the draw.

Drawing prices coverage: how much deposit does this much cover cost, at today's appraisal and today's rate. Releasing does not re-price anything. It hands back the deposit the coverage is carrying, pro rata on the face units let go:

```
capV      = mintsBacked * rho          present value of the outstanding cover
netCap    = capV - mintsPrincipal      net spendable it can release
units     = ceil(amount * mintsBacked / netCap)  face units to deactivate
refund    = mintsPrincipal * units / mintsBacked  the deposit those units carry
```

The distinction only shows once the appraisal moves, and then it is the whole point. A holder who buys cover at one appraisal and releases it at a lower one gets **their whole deposit back**, not the fraction the shrunken cover would cost today.

That follows from what the deposit is. The pool's yield on it funds the premium in perpetuity – that is what makes a policy a one-time purchase rather than a recurring expense – so what the holder actually pays for cover is the *opportunity cost* of the deposit, not the deposit. Re-pricing on the way out would charge them twice for one decline: the pool has already been compensated for holding an over-sized deposit against shrinking cover, because it earned its assumed ROI on the full

deposit the whole time while owing premium only on what was still insured. Keeping the surplus principal as well would be helping itself twice.

The deposit is stored, in `mintsPrincipal[tokenId]`, rather than recomputed. It cannot be recovered from present-day state: it is the sum over past draws of $V_i * \text{effRate}_i / \text{BP}$, and both the appraisal and the premium rate may have moved between them, so `mintsPrincipal / mintsBacked` is a weighted average with no closed form.

Two properties fall out. `capV <= mintsPrincipal` means the position has no spendable left in it – the holder is underwater and must repay before releasing, which is the same rule the post-burn solvency check enforces, reached from the other direction. And when the holder holds no loose BUCK, `netCap == creditLimit - used == balanceOf`: **burning exactly their spendable closes the position and squares the deposit**. A mint and an immediate burn of the same amount cancel exactly; partial releases floor, so the dust stays in the pool until the final release hands it back.

Per-NFT bookkeeping: `mintsBacked[tokenId]` tracks the cumulative outstanding allocation against each NFT, capped at `faceValue`. The invariant `mintsBacked[tid] == activatedValue[tid]` holds at every observation point, kept by two rules: activation happens only inside `_allocateMint` (there is no public `BuckCredit.activate()`), and `updateCredit` may not reappraise below activated coverage. They grow together on mint and shrink together on burn. The burn allocator sizes its unwind from `min(mintsBacked, activatedValue)` anyway – with the rules above they cannot differ, and the clamp is there so that any future divergence costs the system a rounding rather than costing a holder their exit. A mint-burn round-trip on the same NFT order is rate-neutral and not arbitrageable – both sides hit the same `denom`.

The shipped mint sequence:

1. `isVerified(msg.sender)` guard.
2. `buckK.compute()` – advances the PID if `dT` has elapsed; otherwise returns the cached value. Mint and burn together amortize PID work across user activity.
3. Capture `fundingFactor` and – when it is non-zero – the minter’s **pre-activation balanceOf**. The reserve is measured before this mint activates the credit it pledges, so the freshly-activated headroom cannot satisfy its own reserve.
4. `_allocateMint(amount, tokenIds)` reads `BuckCredit.batchCreditInfo(tokenIds)` once, walks the slice array, calls `BuckCredit.activateFromBuck(tid, holder, take_i)` per NFT (grows `activatedValue` and `mintsBacked` in lockstep), returns `(totalCoverage, poolPrincipal)`.
5. **Counter-cyclical funding-factor reserve**, enforced now that `poolPrincipal` is known:

```
require(preActivationBalanceOf >= poolPrincipal * fundingFactor / 1e18,
    "BUCK: insufficient mint funding")
```

The reserve scales with the **insurance principal this mint pays into the pool** – NOT the gross BUCK minted, nor the credit limit applied for. Its job is to put a gentle base-level demand under BUCK exactly when many credit-holders come online to mint (each must hold a little BUCK sized to its insurance cost), not to impede issuance – which a gate on the whole credit limit would. A **zero-premium** credit yields `poolPrincipal = 0` (“zero-cost insurance”), so the requirement is zero and the mint is exempt (the basket / LP bootstrap mints freely). On a non-Carrying account `balanceOf = held + unusedCredit`; `fundingFactor` is 0 under the static controller (gate disabled), ~1.0 at parity, and rises above

1.0 when BUCK is undervalued, so it bites hardest on bootstrapping **insured** credit from a thin reserve while the market already signals too much BUCK.

6. `_accrueJubilee()`, crystallise `insurancePool` and `msg.sender`, debit `poolPrincipal` from the sender's signed raw (`_subBalance` is signed-aware – the raw goes negative if the sender had no positive holdings), credit `poolPrincipal` to `insurancePool`. `_totalSupply` is maintained automatically by `_setBalanceSigned` so that `_totalSupply == sum_a max(0, signedRaw(a))` holds.
7. `Emit Transfer(0, insurancePool, poolPrincipal), Minted(sender, totalCoverage, poolPrincipal, creditValue, buckKValue, newLimit).`

End-state arithmetic for a fresh mint against a zero-balance, non-credit-using holder, where every credit drawn on is **undepreciated**:

<code>signedRawBalanceOf(holder)</code>	<code>= -poolPrincipal</code>	(used = principal paid to pool)
<code>creditLimit(holder)</code>	<code>= sum take_i</code>	(= <code>activatedValue * BUCK_K / UNIT</code>)
<code>balanceOf(holder)</code>	<code>= creditLimit - used</code>	<code>= sum take_i - sum principal_i</code>
		<code>= sum amount_i</code>
		<code>= amount</code> (modulo rounding)
<code>totalSupply</code>	<code>+= poolPrincipal</code>	(only the pool's positive contribution grows)

A depreciated credit delivers in full. The allocator inverts in present insured value and grosses the face units back up by `faceValue / depreciatedFace`, so the last line above holds whatever the appraisal has done. Depreciation costs the holder *capacity* on the credit – the same headroom consumes more face units – never headroom on the mint.

Worked through on a 300,000 structure, LINEAR 200bp/yr on a 60,000 floor, three years in (appraisal 285,600, so $\rho = 0.952$), premium 200bp:

<code>mint(50,000)</code>	<code>-> V</code>	<code>= 62,500</code>	present insured value, <code>ceil(50,000 * 10000/8000)</code>
	<code>principal</code>	<code>= 12,500</code>	<code>= V * 2000/10000</code>
	<code>take</code>	<code>= 65,651.260505</code>	face units, <code>ceil(V * 300,000/285,600)</code>
	<code>balanceOf</code>	<code>= 50,000</code>	exactly what was asked for

The premium is charged on the 62,500 actually insured, not on the 65,651 face slice that carries it: at the assumed 10 % ROI the 12,500 deposit yields 1,250/yr, which is 200bp of 62,500. Charged on the face slice it would have been 1,313/yr, and the holder would have been paying for cover the asset no longer has.

`burn(amount[, tokenId])` mirrors the math but with the most-expensive-first default selector: unwind dearest NFT first, call `BuckCredit.deactivateFromBuck(tid, holder, unwind_i)` which shrinks `activatedValue` and `mintsBacked` in lockstep, debit `poolRefund` from `insurancePool`, credit `poolRefund` to the holder's signed raw (climbs toward zero from used credit, or further positive from a positive balance). `quoteMint` and `quoteBurn` are read-only views that return `(totalCoverage, poolPrincipal)` or `(totalUnwind, poolRefund)` for the same allocator math, so an off-chain optimizer can size required limits and refunds before submitting.

5.3.3 Underwater Is a State, Not a Failure

The burn-side solvency check: after refund and deactivation, the holder's remaining used credit must still fit under the shrunken `creditLimit`, else the burn reverts with BUCK: **post-burn credit used exceeds limit.**

Releasing A units of headroom gives back $A/(1-\text{eff})$ of coverage but refunds only the difference, so the limit always falls faster than the debt does. A holder standing exactly at their limit – the normal end state of mint-then-spend – therefore cannot burn at all until they repay. That is the intended order of operations, and the same one any loan imposes: repay the credit, then release the collateral. Acquiring BUCK and receiving it *is* the repayment; it lifts the signed raw toward zero, and the unwind then fits.

A holder can also be pushed under the line by events they did not choose – a reappraisal, or simply the insured asset depreciating on schedule. Doing nothing about it is a supported outcome, and costs them nothing acutely:

- **The insurance stays in force.** Its premium was paid up front, in perpetuity, and no path cancels it. `activatedValue` does not move.
- **The exposure shrinks on its own.** Jubilee relief accrues against the coverage’s age at $\sim 2\%$ /year, so what closing the position eventually costs declines year on year with no action from the holder. Over ten years a 1,000 BUCK position’s `redeemCost` falls to ~ 800 BUCK.

So there is no liquidation, no margin call, and no forced sale. The position simply becomes cheaper to close the longer it is carried – which is the whole point of the Jubilee.

The `balanceOf(sender) > amount=spendable` check on transfer-out is where credit headroom becomes a real BUCK flow: a transfer larger than `held` but at-or-below `held + unusedCredit` drives the sender’s signed raw negative, and the recipient receives freshly-issued BUCK. `_setBalanceSigned` grows `_totalSupply` by exactly the amount of the positive contribution created on the recipient side (offsetting nothing on the sender side, whose positive contribution just shrank from `held` toward zero or further into used credit).

5.4 Identity-Bound approve()

`approve(spender, amount, E_bob, pi_CP)` is the identity-bound form, and the only one that lays down a receipt fragment. Both `sender` and `spender` must be `isVerified`. The CP proof is required regardless of the spender’s identity flavor (Public-Identity or Encrypted): the always-CP rule produces a uniform per-pair audit record openable on subpoena, and engages the spender’s BN254 identity key for forward security under future identity rotation. Full rationale in *Why approve Is Always CP-Bound*.

On success, `_receiptFragments[sender][spender] = keccak256(E_bob)`, the standard ERC-20 allowance is written, and `markApproved(spender)` freezes the spender’s `isCarrying` flag in the registry (so the carrying flavor consented to cannot retroactively flip). Per-pair cost is $\sim 97\text{K}$ gas (29K CP verify + 46K allowance + 22K receipt SSTORE), paid once. Subsequent transfers between the same pair add only the bilateral check and the receipt event ($\sim 5\text{K}$ above an ERC-20-baseline transfer).

The plain 2-arg `approve(address, uint256)` is *also* supported, and sets nothing but the allowance. This is deliberate, and it costs no privacy: identity is enforced at **transfer** time, never at approve time. A bare allowance only authorises the spender to *initiate* a **transferFrom**; `_identityCheckedTransfer` then independently re-validates both **from** and **to** and the receipt-fragment rule for that pair – and that gate keys on (**from**, **to**), never on (**from**, **spender**). A plain-approved spender therefore cannot manufacture a transfer the rules would not already permit: they still cannot move BUCK between two non-public parties without a real CP receipt fragment laid down by the 4-arg form.

What it buys is that BUCK is a first-class citizen of stock router and Permit2 infrastructure, which requires a plain `approve(PERMIT2, max)`, without weakening any counterparty-privacy invariant.

5.5 `transfer()` and `transferFrom()`

Both route through `_identityCheckedTransfer`, which enforces *mutual decryptability*: every private (non-public) party must hold a per-pair CP receipt fragment for its counterparty, established before the transfer by the 4-arg identity-bound `approve`. Only public-identity contracts may use the deterministic `_identityHash` fallback.

- Requires `isVerified(from)` and `isVerified(to)`.
- For the `to` side (`_receiptFragments[from][to]`): if zero, requires `=isPublicIdentity(from)` — reverts with `BUCK: sender must identity-approve recipient` otherwise. Falls back to `_identityHash(to)` when `from` is public.
- For the `from` side (`_receiptFragments[to][from]`): if zero, requires `=isPublicIdentity(to)` — reverts with `BUCK: recipient must identity-approve sender` otherwise. Falls back to `_identityHash(from)` when `to` is public.
- Both guards pass only when *both* parties are public-identity contracts. EOA-to-EOA transfers require bilateral CP-approve; EOA-to-Public and Public-to-EOA require the private party to CP-approve the public one.

After identity checks, transfer dispatch is automatic on the sender’s `isCarrying` flag: Carrying senders (AMM pools, the Notes pool, the Jubilee fund) take the `_carryingTransfer` path that crystallises both sides through now and apports the sender’s live `buckSeconds` proportionally to the value transferred; Non-Carrying senders take `_nonCarryingTransfer`, which crystallises the sender, checks the amount against `balanceOf(from) = held + unusedCredit`, and shifts raw balance. Because the sender is crystallised immediately before that check, the spendable figure it reads is always settled through the current block. `BuckTransferReceipt(from, to, amount, fromHash, toHash)` is emitted; `_identityHash(a) = keccak256(abi.encode(pk, E_addr))` is deterministic per account.

5.5.1 Reentrancy Guard

Every state-mutating entry point — `transfer`, `transferFrom`, the identity-bound `approve`, both `mint` and `burn` overloads, `mintFromBasket`, `burnFromBasket`, and the delegated-demurrage calls — is wrapped in a transient-storage mutex (EIP-1153; the build targets `cancun`). `TSTORE` / `TLOAD` are 100 gas flat with no cold tier, so the guard costs ~600 gas per guarded call against ~5,150 for the classic storage-slot guard, and it occupies **no storage slot** — the layout is undisturbed. Transient state cannot survive the transaction, so a path that sets the lock and returns without clearing it cannot brick anything.

A bit in the spare `AccountState.flags` word would be cheaper still (~390 gas, since the slot is written anyway) and was measured and rejected. It guards the wrong thing: an attacker re-enters from whatever address they like, so locking `msg.sender` stops nothing, and locking every account an operation touches means publishing a lock `SSTORE` per account before every call-out. The bit would also share a slot with the balance, and this contract’s idiom is read-struct-into-memory / write-struct-back — every such sequence silently clears a lock set after the read.

The guard has **no exemptions**. It needed one when `BuckCredit` called back into `Buck` through the credit-limit hook – that call arrives from inside the very `activateFromBuck` a guarded mint is executing, so guarding it deadlocked every mint. With the hook gone, the call graph between the contracts runs one way and the question does not arise. The plain 2-arg `approve` is left unguarded on purpose: it touches only the allowance map, calls nothing, and is the hottest entry point for router integration.

5.6 Demurrage and Jubilee

BUCK accrues continuous demurrage to fund a perpetual Jubilee. Each account’s `buckSeconds` field is the cumulative integral of $(\text{positive raw balance} \times \text{dt})$ crystallised at `timestamp`; the live fee owed is

```
feeOwing(a) = (buckSeconds + max(0, balance) * (now - timestamp)) * BASE_RATE_PER_SEC / SCALE
              = 0      when signed raw <= 0    (no demurrage on used credit)
```

with `BASE_RATE_PER_YEAR = 0.02` (2%/year) and `SCALE = 1e27`. Demurrage is a per-second property of *held* BUCK; accounts using credit (`signed raw < 0`) accrue zero fee until they climb back into positive territory. `_crystallize` uses `max(0, signedRaw)` as the rectangle height so a negative-going transfer doesn’t poison the `buckSeconds` integral. This makes sense; any account extends into its credit only when a transfer has produced a positive BUCK balance somewhere else, where it accrues demurrage.

Visible balance:

- **Non-Carrying** (default for EOAs): `balanceOf(a) = held + unusedCredit` where `held = max(0, signedRaw - feeOwing)` and `unusedCredit = max(0, creditLimit - used)`. The locked fee stays inside the account’s own slot; spendable on the held side shrinks over idle periods; unused credit shrinks if `BUCK_K` falls.
- **Carrying** (AMM pools, Notes, Jubilee, etc.): `balanceOf(a) = max(0, signedRaw)`. The account never loses visible balance to demurrage; instead, on every outflow the value-weighted age is folded into the recipient’s `buckSeconds` so the locked fees ride with the BUCKs. Carrying accounts hold no `BuckCredit` NFTs and never go negative (the spendable check inside `_carryingTransfer` rejects any transfer that would drive `raw < 0`).

`_crystallize(a)` folds the positive time rectangle into `buckSeconds` and bumps `timestamp`; it never changes balance and tolerates negative `signed raw` (rectangle = 0). Carrying transfer crystallises both sides through now, apportions `liveBs * value / raw` of the sender’s live integral to the recipient (where `liveBs = buckSeconds + raw * (now - timestamp)`), and settles each side in one `SSTORE`.

The Jubilee is a normal account at `address(this)`, treated as Carrying. On every mint or burn, `_accrueJubilee` writes

```
_state[address(this)].balance += totalSupply * BASE_RATE_PER_SEC * elapsed / SCALE
```

directly into Jubilee’s slot via a raw slot write – bypassing `_setBalanceSigned` – so `totalSupply` is **not** mutated. Demurrage is internal redistribution from positive-balance holders into the system-wide Jubilee, not fresh minting. The exact invariant:

```
sum_a max(0, signedRawBalanceOf(a)) == totalSupply + jubileeActual()
```

Jubilee is the system-wide fee escrow. Every BUCK has exactly one fee owner: non-Carrying accounts lock it silently inside their raw balance (`balanceOf = raw - feeOwing`); Carrying accounts (Notes pool, AMM pools, Jubilee itself) show `balanceOf = raw`, so their fee portion sits in Jubilee instead. The `_accrueJubilee` call credits Jubilee at the full `totalSupply` rate, covering Carrying holdings too. When Carrying BUCKs flow to a non-Carrying account via `_carryingTransfer`, the recipient absorbs `liveBs * value / raw` as extra `buckSeconds`, where `liveBs` is the sender's full live integral (crystallised history plus the current elapsed rectangle). Jubilee pre-accrued the full fee at the `totalSupply` rate; it now debits in exact proportion to the BUCKs transferred, with both sides settled through the current block. `JubileeAccrued(delta, newBalance)` surfaces every accrual.

The Jubilee is the sink that funds default coverage, perpetual-ground-rent disbursement, or a periodic balance reset – all governance-driven on-chain disbursement paths land on top of this single accumulating slot.

5.6.1 Delegated Demurrage: a Designated Fee Payer

An account may route its demurrage to a designated payer, so that one Identity's several accounts concentrate their erosion in one place instead of each being eaten from underneath.

The mechanism is a **transfer of buck-seconds**, not a discount. Demurrage here is a lien, never a movement: the fee is locked inside the account's own raw balance, and the aggregate of those liens is what backs the Jubilee's accrual against `totalSupply`. Since `sum_a buckSeconds(a)` tracks `integral(totalSupply dt)`, destroying buck-seconds anywhere would leave the Jubilee accruing against nothing. So at crystallisation the sponsored account's (`balance * dt`) rectangle is moved into the payer's slot rather than its own. The total is conserved exactly, and **no raw BUCK moves at all** – no supply invariant is touched by the feature.

Absorption is capped at the payer's own lien capacity: the point at which `feeOwing(payer)` would exceed `rawBalanceOf(payer)`. Past that the lien would be uncollectible, which is precisely how delegation would have become a way *out* of demurrage rather than a way to relocate it. Whatever the payer cannot carry stays with the sponsored account, so the worst case is "no delegation at all" – never uncollected demurrage.

Consent is two-sided. `requestDemurragePayer(payer)` elects; `acceptDemurragePayer(account)` takes on the liability; either side may `clearDemurragePayer`. The payer's consent is what stops anyone dumping unbounded fee exposure onto any balance. Neither party may be Carrying – a Carrying account's `balanceOf` ignores its fee entirely, so its lien would not bite – and chains are refused, since routing is one hop by construction and a chain only makes "who is actually paying" unanswerable by inspection.

Settlement is lazy, on the sponsored account's next touch, which is the cadence at which BUCK accrues everything else. Between touches the payer's own `feeOwing` does not yet include its sponsees' pending rectangles – finding them would mean enumerating sponsees – so `settleDemurrage(account)` is a permissionless poke that forces the transfer. No spend decision ever uses a stale number: every balance-moving path crystallises the account it is about to debit immediately before reading its balance.

Cost: nothing for accounts that do not opt in, since dispatch is a mask on the `flags` word the transfer path already holds in memory; +863 gas warm and +8,434 cold for those that do. Conservation is asserted against identical unsponsored control pairs; the residual is at most one raw unit (1e-6 BUCK) per account merged, from flooring once over a pooled slot instead of twice, and it always lands on the system's side of the ledger.

5.7 Integrating with BUCK

BUCK is a standards-compliant ERC-20 and works with stock routers, pools and Permit2. Three of its properties are unusual enough that an integration written against ordinary tokens should be checked against them.

Identity is enforced on transfer, and both sides must be registered. A transfer to an unregistered address reverts. Any contract intended to hold BUCK must be bound via `IdentityRegistry.bindContract` – typically with `isPublicIdentity_ = true, isCarrying = true` for a pool or vault (see `IdentityRegistry`). This is the one change most integrations need: deploy, then bind, before the first transfer in.

balanceOf moves on its own. For a non-Carrying account it is `held + unusedCredit`, and the credit half is a live function of the holder’s insured assets and of `BUCK_K`. It falls as those assets depreciate, moves when an insurer reappraises one, and moves when the PID moves `BUCK_K` – all without any transfer occurring. Demurrage moves the held half downward continuously for the same reason. Consequences:

- Do not treat a stored `balanceOf` snapshot as still valid in a later block.
- Do not derive a share price or a collateral ratio from an account’s `balanceOf` unless you intend it to track that account’s insured assets. For custody questions use `rawBalanceOf` / `signedRawBalanceOf`, which report only BUCK actually held.
- Do not sample `balanceOf` from inside a callback you do not control: views are not covered by the reentrancy guard, and the ordinary read-only-reentrancy discipline applies.

Contracts bound as **Carrying** are exempt from all of this: `balanceOf` is exactly the raw balance, no credit and no demurrage deduction. Every pool in this system is Carrying, which is why nothing in-tree is affected.

Transfers may exceed balanceOf at the moment you read it, and may exceed held BUCK. A transfer at or below `held + unusedCredit` succeeds by driving the sender’s signed raw negative and issuing fresh BUCK to the recipient. A pre-flight `require(balanceOf(x) >= amt)` in your own contract is therefore redundant, and against a stale read it is wrong in both directions. Let the transfer revert instead.

6 BuckBasket: Direct-Mint Orchestrator + USD-Free Value Anchor

BuckBasket is the contract that closes the loop between BUCK and the real-world commodity tokens it tracks. It registers a set of TOKEN/BUCK Uniswap-V3 pools (one per basket constituent), custodies a single full-range Buck-owned LP position in each, lets anyone direct-mint BUCK by depositing a constituent (or recycle already-minted BUCK into the most underweight constituent), and on redemption pulls BUCK out of overweight pools while routing the surplus into the most underweight pool as treasury LP. It is also the publisher of the USD-free process variable read by `BuckKControllerDirect`.

6.1 Two Kinds of BUCK

BUCK in circulation today has two provenances, both flowing through the same ERC-20:

1. **BuckBasket-minted BUCK** – minted by `BuckBasket` via the privileged `Buck.mintFromBasket(to, amount)` hook when a depositor pledges a `TOKEN`. Tracked in `BuckBasket.totalOutstandingBuck`; burned on redemption. The activation gate is *not* the `BuckCredit` credit limit – it is the depositor’s willingness to put `TOKEN` (or `BUCK`) into the basket at the current pool valuation.
2. **Externally-supplied BUCK** – created by the standard `Buck.mint(amount, tokenIds)` path described in `BUCK`, backed by `BuckCredit` insured-asset NFTs and gated by `BUCK_K`. These `BUCK` reach the `TOKEN/BUCK` pools indirectly via arbitrage swaps and are the lever the PID actually pulls on.

The two supply paths converge in the basket’s `BUCK`-side balance. Price-up `TOKEN` moves leave extra `BUCK` in the pool (`TOKEN` bought with `BUCK` by external traders); price-down moves draw `BUCK` out. Combined with reinvested treasury `BUCK` from prior redemptions, the basket’s NAV typically exceeds `totalOutstandingBuck` – the spread is what compounds for depositors and underwrites the "profit" side of the redemption split.

6.2 Constituents and Weights

Governance registers each basket constituent via `addBasketToken(token, decimals, initialPriceInBuck, weightBp, feeTier)`. `weightBp` is the declared target weight in basis points; passing 0 defaults to an equal share (10000 / N). The call:

1. Renormalizes every existing constituent’s `targetWeightBp` so the total across the N+1 constituents sums to 10000 bp, preserving each existing constituent’s **relative** weight ratio.
2. Re-prices every existing constituent’s `basketAmount` at the **current** pool spot price so its contribution to `basketValueInBuck` equals its declared weight share of 1.0 `BUCK`.
3. Finds or creates the `BUCK/TOKEN V3` pool at the requested fee tier, initializes it at `initialPriceInBuck` (silently no-ops if already initialized), and bumps the observation cardinality so TWAP reads are available immediately after the first swap.
4. Computes the full-range tick bounds for the pool’s fee-tier spacing and stores the constituent’s `buckIsToken0` ordering.
5. Calls `controller.reprime()` so the controller absorbs the dilution-discontinuity in `basketValueInBuck` without firing a one-cycle P/I spike.

A future `rebalanceWeights(tokens[], weightBps[])` will let governance re-weight (or, with `weightBp = 0`, remove) one or more constituents in a single call – recomputing all `basketAmount` values from the current spot prices so each constituent’s contribution again matches its declared share of 1.0 `BUCK`.

6.3 Process Variable: `basketValueInBuck()`

```
basketValueInBuck = sum_i ( basketAmount_i * pool_price_in_BUCK_i / 1e18 )
```

This is the direct-embodiment "process variable" – the value of one basket bundle measured in `BUCK` – and is what `BuckKControllerDirect` reads. TWAP-windowed reads (`twapWindow` seconds) fall back to spot if the pool is too cold to satisfy the request, so the PID degrades gracefully on a freshly-added constituent rather than freezing.

6.4 Direct-Mint: depositToken

`depositToken(token, tokenAmount, maxDeviationBp)` has two paths chosen on `token`:

- **Basket TOKEN deposit**: the depositor pledges a registered constituent. `BuckBasket` reads the pool's spot price, mints `tokenAmount * spotPrice` worth of BUCK to itself via `Buck.mintFromBasket`, and LPs the (TOKEN, BUCK) pair as full-range liquidity in that constituent's pool. An ERC-721 receipt (`BuckBasketReceipt`) records the (BUCK principal, TOKEN principal, original token, deposit time).
- **Already-minted BUCK deposit** (`token = address(buck)=`): the depositor's BUCK is pulled in, swapped on the **most underweight** pool for that pool's TOKEN, partner BUCK is freshly minted against the received TOKEN, and the full (TOKEN, BUCK) pair is LP'd into the underweight pool. This is the cross-pool rebalance arrow on the deposit side.

`maxDeviationBp` optionally guards against TWAP/spot divergence (set to 0 during bootstrap or cold-pool deposits; non-zero rejects the deposit if $|\text{spot} - \text{TWAP}| > \text{maxDeviationBp} / 10000$).

6.5 Redemption: "Sell High, Recycle to Buy Low"

`redeem(receiptId, redeemBp, maxDeviationBp)` settles a value claim `valueClaim = redeemBuck * NAV / totalOutstandingBuck` where NAV is the basket's total LP value across all pools. The claim is split across constituents by `_allocateRedemption`, then settled in seven phases:

1. **Validate + size** the receipt; compute `redeemBuck` as either the full `buckPrincipal` or a basis-point fraction.
2. **Slippage guard** on the depositor's original pool (the known gap: this guard should also fire on every pool actually withdrawn from – tracked as BUG #5).
3. **Allocate** `redeemValue` across pools. Each pool's "ideal" allocation is $\text{alloc}_i = v_i - t_i * (\text{NAV} - \text{redeemValue})$ where t_i is the pool's natural value share at target. Heavily overweight pools contribute *more* than their proportional share; heavily underweight pools clamp to zero. Two passes handle the equilibrium case (proportional) and the rare "must dip into underweight" case. A small-redemption fast path takes the entire claim from the most overweight pool when `redeemValue` is below `SMALL_REDEEM_BP` (1 %) of that pool's value, saving the multi-pool gas in the dominant case.
4. **Burn LP** from each allocated pool. TOKEN and BUCK are held inside `BuckBasket`; no transfers yet, so any aggregate BUCK shortfall (e.g. from a single pool dominating the allocation but carrying mostly TOKEN) can still reach back into the basket's TOKEN holdings.
5. **Cover shortfall** by greedy TOKEN→BUCK swaps on the pools with the most TOKEN balance left, using exact-input swaps padded by `SHORTFALL_BUFFER_BP` (2 %). Any residual gap below `MAX_ORPHAN_DUST_WEI` (1 μ BUCK) is tolerated; larger gaps revert with "pool depth too thin".
6. **Pay depositor** the surviving TOKEN side per pool, emit one `RedeemedFromPool` event per touched pool (TOKEN-to-user, TOKEN-swapped, L burned).

7. **Reinvest profit** above the `MIN_REINVEST_BUCK` floor (0.001 BUCK): the BUCK side minus the principal burn is routed through `_buckToLp` into the most underweight pool’s TOKEN and LP’d as a treasury position. Sub-floor profit stays in the basket’s BUCK balance and is swept by the next redemption (still treasury-owned, just not yet LP’d). Then `Buck.burnFromBasket` burns the principal, `totalOutstandingBuck` decrements, and the deposit is finalized (cleared on full redemption; both BUCK and TOKEN principals scale down proportionally on partial).

The depositor/treasury split is **approximately** 50/50: full-range V3 LP is roughly 50/50 by value at any price, so the TOKEN side (depositor) and BUCK side (treasury) are roughly equal. The principal burn comes out of the BUCK side, slightly favouring the depositor.

6.6 Invariant

After all deposits exit, `totalOutstandingBuck` = 0 and remaining LP value is entirely treasury-owned – compounded TOKEN appreciation, accumulated AMM fees, and external-arb BUCK that didn’t get claimed. Modulo bounded `MAX_ORPHAN_DUST_WEI` per-redemption dust, this is the liveness backstop the audit trail rests on.

6.7 Known Gaps

The contract carries BUG #N: tags for the redesign work still in flight:

- #5 – Slippage guard runs on the depositor’s original pool, not on the pools actually withdrawn from.
- #6 – `_buckToLp` / `_swapTokenForBuckExactIn` swaps use the permissive `MIN/MAX_SQRT_RATIO ± 1` price limit; should be TWAP-bounded to resist sandwich.
- #7 – `_mostUnderweightPool` / `_mostOverweightPool` read spot prices; should use TWAP.
- #8 – `_reinvestBuck` LPs into a single most-underweight pool; should mirror the redemption’s proportional allocation.
- #9 – TOKEN deposits LP into the deposited token’s own pool with no underweight-routing – asymmetric with the BUCK-deposit path.
- #10 – `Buck.mintFromBasket` bypasses `BuckK’s fundingFactor` gate; document or fold basket-minted BUCK into the PID’s accounting (see Dynamic Issuance below).
- #11 – `_reinvestBuck` emits no event; downstream observers can’t distinguish treasury reinvestment from external arb trades.
- #13 – Late-basket-life redemptions can revert `pool depth too thin` when the basket NAV is too low to cover the remaining outstanding (genuine liquidity exhaustion, not dust); a graceful tail-redemption mode is future work.

6.8 Dynamic Issuance: Why BuckBasket Needs BUCK_K

`BuckBasket` mints BUCK at **exactly** the current pool spot price. That is value-neutral at the moment of deposit, but it is also the cause of a structural deflationary pressure on BUCK:

- Direct-mint adds new BUCK only when a depositor **wants** to LP a constituent. External BUCK demand (somebody wants to buy a TOKEN and the cheapest path is BUCK -> TOKEN swap on the BuckBasket pool) does **not** mint new BUCK – it just rearranges existing BUCK between the pool and the swap-caller.
- The only path that produces "free" BUCK against assets **outside** the basket is `Buck.mint(amount, tokenIds)` via `BuckCredit` NFTs, gated by `maxLimit = totalCreditValue * currentBuckK / 1e18`. When external arbitrageurs buy TOKEN with BUCK from the basket pools, BUCK gets scarce (price up vs. TOKEN each TOKEN/BUCK pool quotes a higher TOKEN price in BUCK terms `basketValueInBuck` rises above 1.0).

Sign convention (inherited from BUCK_K):

- `basketValue > 1.0` (BUCK undervalued / inflation: TOKEN expensive in BUCK): `error < 0` `buckK decreases` `BuckCredit` backed mints shrink BUCK supply contracts basket value falls back toward 1.0.
- `basketValue < 1.0` (BUCK overvalued / deflation: TOKEN cheap in BUCK): `error > 0` `buckK increases` `BuckCredit` backed mints expand BUCK supply grows basket value rises back toward 1.0.

So the closed-loop equilibrium picture is: `BuckBasket` establishes the TOKEN-side anchor and publishes the deviation signal; `BuckKControllerDirect` integrates that deviation into a credit-limit multiplier; `Buck` meters new supply against the multiplier; external arb agents close the loop by moving BUCK between the basket pools and the wider market. Direct-mint deposits are the **value injector** (asset coming in); `BuckCredit` backed mints under PID control are the **supply regulator** (BUCK going out).

7 BUCK_K: Value Stabilization Controller

BUCK_K supplies the credit-limit multiplier `BUCK.mint` reads to size against `totalCurrentValue`. The `IBuckK` interface is three functions – `currentBuckK()` external view returns (uint256), `compute()` external returns (uint256), and `fundingFactor()` external view returns (uint256) (counter-cyclical insurance scalar) – so BUCK is invariant to which implementation is wired in. `BUCK.mint` and `BUCK.burn` both call `compute()` so user activity drives the controller and amortises any PID work; the function is otherwise permissionless, keepers and individual users can poke it directly during quiet stretches. Three implementations ship today: `BuckKControllerStatic` (governance-set constant), `BuckKControllerDirect` (USD-free PID against `BuckBasket`), and `BuckKController` (PID against an external four-pool USDT/USDC basket of Uniswap-V3 TWAPs). The PID mechanics – prime cycle, `dS` correction, `dT` / `dTMax`, anti-windup, `fundingFactor()` – live in a shared `BuckKControllerBase` abstract contract; each concrete variant only overrides `_readReferences()` returns (int256 `buckValue`, int256 `basketValue`) to wire its specific price sources.

7.1 Static Variant

`BuckKControllerStatic` holds one uint256 `buckK` (18 decimals, `1e18` = neutral). Functions: `currentBuckK()`, `compute()` (returns the stored value, non-view to match the `IBuckK` signature), `setBuckK(uint256)` (governance-only), `transferGovernance(address)`. This is the simpler choice for early deployments where no BUCK price oracle exists yet, exercising the identity, demurrage, and Notes layers while BUCK_K is held constant.

7.2 Direct Variant (USD-Free)

`BuckKControllerDirect` is the embodiment used in tandem with `BuckBasket`. It has no external stablecoin intermediary; BUCK measures itself against a basket of TOKEN/BUCK Uniswap-V3 pools that `BuckBasket` owns and prices.

PID Term	Source	Meaning
Setpoint	UNIT (= <code>1e18</code> , constant by definition)	One BUCK should buy one basket b
Process Variable	<code>BuckBasket.basketValueInBuck()</code>	<code>sum_i (basketAmount_i * pool_pri</code>
Error	<code>buckValue - basketValue = 1.0 - basketValue</code>	Positive: BUCK overvalued; negative
Output (BUCK_K)	<code>1e18 + (P*Kp + I*Ki + D*Kd)/1e18</code>	Credit-limit multiplier (1.0 = neutra

The sign convention is identical to the External-Basket variant below (all shipped gains are positive), but the error semantics are inverted relative to a "BUCK price in USDT" reading:

- `basketValue > 1.0` (one BUCK now buys **less** than one basket – BUCK undervalued / inflation): `error < 0` BUCK_K *decreases* credit contracts supply shrinks BUCK recovers value vs. the basket.
- `basketValue < 1.0` (one BUCK now buys **more** than one basket – BUCK overvalued / deflation): `error > 0` BUCK_K *increases* credit expands supply grows BUCK gives back value to the basket.

`basket` is wired one-shot from governance via `setBasket(address)` (immutable thereafter); deploying a new `BuckBasket` requires a new controller. `reprime()` external is callable **only** by the registered `BuckBasket`; `BuckBasket.addBasketToken` invokes it after every constituent addition so

the dilution discontinuity in `basketValueInBuck` – a new constituent’s `basketAmount * spotPrice` contribution appears in a single block – doesn’t manifest as a one-cycle P/I spike. `_reprime()` recaptures P against the new process state and re-derives I so the next no-error cycle reproduces the current `buckK`.

Cold-pool / pre-basket behaviour: `_readReferences` returns (UNIT, UNIT) until `setBasket` fires, so the controller returns the priming `buckK` and runs no PID math. Once the basket is wired and observations exist, the loop is identical to the External-Basket variant – only the PV source differs.

7.3 External-Basket Variant

`BuckKController` is the original Uniswap-V3-basket variant. It reads BUCK and a multi-token basket from separate Uniswap-V3 pools (typically priced in USDT/USDC) and treats their difference as the error term; it predates `BuckBasket` and remains in tree for deployments where the basket-of-pools approach is preferred over (or runs in parallel with) the direct embodiment. It follows the `ezpwd::pid` controller layout, ported to 18-decimal Solidity fixed-point.

PID Term	Source	Meaning
Process Variable	BUCK/USDT Uniswap V3 TWAP	What a BUCK actually trades for
Setpoint	Commodity-basket TWAP sum	What a BUCK should be worth
Error	Process - Setpoint	Positive: BUCK overvalued; negative: undervalued
Output (BUCK_K)	$1e18 + (P*Kp + I*Ki + D*Kd)/1e18$	Credit-limit multiplier (1.0 = neutral)

Sign convention (all shipped gains are **positive**):

- **BUCK below basket (inflation):** `error < 0` -> `BUCK_K decreases`. Credit contracts; holders running near their limit must burn BUCK to stay compliant; supply shrinks; BUCK price recovers toward basket from below.
- **BUCK above basket (deflation):** `error > 0` -> `BUCK_K increases`. Credit expands; holders can mint additional BUCK against their NFTs and sell it into the pool at the premium; supply grows; BUCK price recovers toward basket from above.

Two canary tests in `test/BuckKController.t.sol` guard this direction (`test_sign_convention_inflation_contract` and `test_sign_convention_deflation_expands_credit`); any sign flip in the controller surface fails them within one PID cycle. See alberta-buck-ethereum-example.org for the bootstrap, equilibrium, and arbitrage examples that exercise the loop end-to-end.

7.3.1 External Basket and BUCK Price Oracles

The External-Basket controller reads basket components from two parallel sources, both summed into the basket cost: a Chainlink-feed array (`BasketComponent { feed, weight, feedDecimals }`, used in tests against mock aggregators) and a Uniswap-V3-pool array (`BasketPool { pool, baseToken, quoteToken, weight, baseDecimals, quoteDecimals, twapInterval }`). `addBasketComponent` and `addBasketPool` are governance-only. The shipped target basket is V3-pool-sourced:

Pool	Base	Quote	Weight
XAUT / USDT	XAUT (6d)	USDT	25 %
PAXG / USDC	PAXG (18d)	USDC	25 %
cbBTC / USDC	cbBTC (8d)	USDC	25 %
WBTC / USDT	WBTC (8d)	USDT	25 %

Two gold tokens and two BTC tokens, paired against two stablecoins, gives the basket resilience against any single asset-token or basis-currency freeze. `setBuckPriceOracle(pool, buckToken, quoteToken, quoteDecimals, twapInterval)` configures the BUCK price oracle (typically BUCK/USDT).

Pool reads route through `UniswapV3OracleLib (src/lib/)` – a 0.8-compatible port of the `OracleLibrary` `/=TickMath=` `/=FullMath=` helpers we need. The vendored `lib/v3-periphery OracleLibrary` is pinned to `<0.8.0` and can't be imported directly under `0.8.20+`. When a pool's `twapInterval > 0` the controller calls `consult(pool, secondsAgo)` for the time-weighted mean tick; when zero it reads `slot0().tick` directly (test-only).

7.3.2 Priming and dS Compensation (shared with the Direct Variant)

The first `compute()` after deployment captures references and returns the cached `buckK` without running PID math:

```
prime cycle: P = current_error; lastBasketCost = current_basket; primed = true;
              return buckK
```

This avoids the spurious first-cycle derivative spike that arose from `P_prev = 0` (default storage) combined with a non-zero initial error. All subsequent cycles compute

```
dS = basketCost - lastBasketCost
D  = (error - P_prev - dS) * UNIT / dt
```

where `dS` captures setpoint movement (gold spike, governance basket re-weight, oracle update). Subtracting `dS` from `(error - P_prev)` strips out target movement so derivative reflects only BUCK's drift relative to its (moving) target. Without `dS`, a basket-side jump would otherwise fire `Kd * basketJump / dt` as if BUCK had moved, slamming `BUCK_K` toward a rail.

7.3.3 dT, dTMax, and Pokability (shared with the Direct Variant)

Three time constants govern when and how aggressively the controller responds:

- **dT (minimum cycle interval):** `compute()` short-circuits if `now - lastUpdate < dT` and returns cached `buckK` as a one-SLOAD operation. This is the amortisation knob – multiple mints in a single `dT` window share one PID cycle's gas cost. 60 s is reasonable for a chain with ~12 s blocks (one PID cycle per ~5 blocks).
- **dTMax (long-gap clamp):** when `now - lastUpdate > dTMax`, the cycle integrates as if only `dTMax` seconds had passed. Real `lastUpdate` still advances to `now`, so subsequent cycles measure forward correctly. Default `type(uint256).max` (no clamp); governance opts in via `setDTMax`. Typical value 1 h: a 24-h silence then integrates as one 1-h step rather than slamming the integral with 24 h of accumulated error.
- **Pokability:** `compute()` is permissionless and side-effect-only-when-due. In normal operation `Buck.mint / Buck.burn` drive it; in long quiet stretches a keeper can poke it directly to keep the integral fresh, or rely on `dTMax` to bound the eventual catch-up step.

The TWAP window itself (currently 600 s on every pool) is the manipulation-resistance knob. A flash-loan-shape attack that drives BUCK to \$0.50 in one block and unwinds the next leaves spot collapsed but TWAP unmoved – the test suite verifies that during such an attack the TWAP read sits at ~\$1.000002 while spot reads \$0.500.

7.3.4 Future: Amortizing PID Work via a State Machine

Today’s `compute()` reads all five pools and runs the full P/I/D math in a single call. At ~5 V3 pool reads + a few mults this is cheap (~300-500 K gas including a state write). If a richer PID grew expensive enough that mint-time amortization became painful, the cycle work could be split into a state machine over multiple calls – each transfer advancing one micro-step of (read pool *i*, accumulate, finalize). The tradeoff is that pool reads stretch over real time and so the basket sample becomes incoherent unless every input is itself TWAP’d over a window covering the state-machine duration; today’s full-cycle compute wins for that reason. Revisit when oracle reads grow or the basket expands materially.

7.3.5 Tuning (shared with the Direct Variant)

Gain	Effect of increasing	Risk
Kp	Faster response to deviations	Oscillation, over-correction
Ki	Eliminates steady-state error	Integral windup, slow recovery
Kd	Damps oscillation, anticipates trends	Noise amplification

Initial gains should be conservative (high Kd relative to Kp, low Ki). `dT` acts as a coarse damper, `dTMax` as a backstop. Priming and `dS` compensation make non-zero Kd safe at small `dT`: pre-priming, every fresh deploy with `Kd > 0` had a permanent derivative contribution because `P_prev` was stuck at 0 – post-priming, `P_prev` captures the actual initial error and the derivative dies off as soon as the system reaches a new equilibrium.

A future enhancement is on-chain gain scheduling: maintain a rolling variance of the error signal and rotate between conservative and responsive gain sets at regime boundaries. This maps to the `ezpwd::pid::reset()` hot-swap pattern in the C++ reference.

8 Notes: Privacy-Preserving Denomination

BUCK transfers carry a public (`from`, `to`, `amount`) quartet. Notes lift values, flavours, and identity bindings off the public ledger by escrowing BUCK in a commitment pool: holders post Poseidon-hash commitments via a Groth16 mint, then redeem any commitment later via a second Groth16 spend with a fresh nullifier. The contract’s public state is just a list of opaque 32-byte commitments, a 30-slot recent-roots ring buffer, a nullifier set, and a running face-value total. Full design in Notes and Proofs; this section documents the Ethereum surface only.

8.1 Mint Circuit (Batch, In-Circuit Merkle Insertion)

`circuits/mint_batch.circom` is a parameterised batch-mint circuit. Each pinned size *N* (specialised at $N \in \{1, 2, 4, 8, 16, 32\}$, with the smaller sizes useful for unit tests and small wallet flows) produces its own `MintBatchN*Groth16Verifier.sol`. The shipped `Notes` contract holds a single `mintVerifier` adapter; per-*N* dispatch lives inside the adapter (a single `IMintVerifier` chooses the right pinned verifier by `cms.length`). Public-signals arity is 5 regardless of *N*:

Signal	Description
<code>oldRoot</code>	Live note Merkle root the prover read from chain
<code>newRoot</code>	Root after folding <code>cm[0..N-1]</code> starting at <code>nextLeafIndex</code>
<code>nextLeafIndex</code>	Live tree size the prover read from chain
<code>totalFace</code>	Sum of note values, range-bounded to 128 bits
<code>cmBatchHash</code>	<code>keccak256(abi.encodePacked(cm[0..N-1]))</code> – ties calldata to the SNARK

Per leaf, the circuit enforces a Poseidon-5 commitment opening `cm_i == Poseidon(flavor_i, v_i, rho_i, idHash_i, predicate_i)`, `Num2Bits(128)` range bounds on `v[i]` and `totalFace` (closing field-overflow attacks), one sum check `totalFace == sum v[i]`, and an $N \times \text{TREE_DEPTH}$ Poseidon-2 dual Merkle walk: `priorWalk` consumes `ZERO_VALUE` at the insertion slot to attest `oldRoot`, `postWalk` consumes `cm[i]` to advance `rollingRoot` to `newRoot`. Both walks share prover-supplied siblings. A keccak hash check binds `cmBatchHash` to the per-leaf commitments. $\sim 6\text{K}$ R1CS per leaf ($\sim 1.7\text{K}$ opening + $\sim 4.3\text{K}$ insertion).

Verifier bytecode sizes: 14.5 KB at $N=16$, 20.8 KB at $N=32$ – both under the EIP-170 24 KB ceiling. Per-mint gas: 461K at $N=1$ (constant $\sim 216\text{K}$ Groth16 verifier dominates), 624K at $N=16$ ($\sim 39\text{K}/\text{leaf}$ – the shipped sweet-spot), 1.12M at $N=32$. Asymptotic per-leaf cost is $\sim 31\text{K}$. Promotion to $N \geq 64$ needs one of the three escape hatches in `alberta-buck-notes-rollup-mint.org` (`calldata-IC` hash-pinned, sharded `SSTORE2`, plain `SLOAD`), with `calldata-IC` the natural fit for the Holochain wallet.

The trusted setup in `scripts/snark/setup.sh` uses a fixed dev-only entropy string. Production would run a multi-party ceremony per pinned N ; the script is the scaffolding.

8.2 Spend Circuits (B-flavor, A-flavor V2)

`circuits/spend.circom` proves redemption of a single B-shape note under an on-chain Merkle root. Public inputs [`noteRoot`, `nullifier`, `face`, `recipient`, `chainId`]; private witness (`flavor`, `v`, `rho`, `idHash`, `predicate`, `pathElements[20]`, `pathIndices[20]`). Constraint groups: `Num2Bits(128)` on `face` and `v`, `cm == Poseidon-5(flavor, v, rho, idHash, predicate)`, Tornado-style depth-20 Merkle inclusion under `noteRoot`, B-tag nullifier `Poseidon-3(rho, idHash, 4242)`, and value conservation `face * 1 == v * 1` with a quadratic anchor. $\sim 12,000$ R1CS. `SpendGroth16Verifier` takes the 5-element public-signals tuple; `SpendVerifierAdapter` presents the narrow `ISpendVerifier`.

`circuits/spend_a.circom` V2 is the A-flavor (identity-bound) sibling. Public inputs grow to [`noteRoot`, `nullifier`, `face`, `recipient`, `chainId`, `E_n.R.x`, `E_n.R.y`, `E_n.C.x`, `E_n.C.y`] (9 total). Adds the A-tag nullifier `Poseidon-3(rho, idHash, 4243)`, a flavor-in- $\{1,2\}$ gate, and an in-circuit Poseidon-8 binding gate `Poseidon-8(E_n, issuerData) == idHash` (~ 265 R1CS) that pins the publicly-revealed `E_n` to the leaf at mint time. Total $\sim 13,300$ R1CS.

The cryptographic identity gate – proving the spender holds a single `sk_dep` that opens both `E_n` and the registered `E_reg` – lives in `IdentityRegistry.verifySpendCP` rather than the circuit. In-circuit BN254 G1 arithmetic over the BN254 scalar field would have cost $+250\text{K}$ R1CS per group operation; off-chain CP-DLEQ verified via EIP-196 precompiles costs $\sim 36\text{K}$ gas instead. `Notes.spendACP` calls the Groth16 verifier and `verifySpendCP` atomically; both must succeed. The DLEQ is a 4-element Sigma proof:

Prover picks t in Fr , computes $T1 = t \cdot G$, $T2 = t \cdot (R_{\text{reg}} - R_n)$. Challenge: $e = H(R_n, R_{\text{reg}}, C_n, C_{\text{reg}}, pk_{\text{dep}}, T1, T2, \text{recipient}, \text{chainid})$. Response: $s = t + e \cdot sk_{\text{dep}}$. Proof: $(e, s, T1, T2)$ – 4 field elements, $\sim 36\text{K}$ gas to verify on-chain.

The invariant the two layers together enforce: A-note loss-recovery via identity re-issuance is impossible by design. `sk_rec` must be backed up like a hardware-wallet seed; loss is terminal. Companion documents `Notes` and `Notes Flow` document the wallet-side implications.

8.3 Verifier Interfaces

`Notes` is decoupled from the SNARK toolchain via narrow interfaces:

```

interface IMintVerifier {
    function verifyMint(bytes proof, uint256 oldRoot, uint256 newRoot,
        uint256 nextLeafIndex, uint256 totalFace,
        uint256[] calldata cms) external view returns (bool);
}
interface ISpendVerifier {
    function verifySpend(bytes proof, uint256 root, uint256 nullifier,
        uint256 face, address recipient, uint256 chainId)
        external view returns (bool);
}
interface ISpendAVerifier {
    function verifySpendA(bytes proof, uint256 root, uint256 nullifier,
        uint256 face, address recipient, uint256 chainId,
        uint256 enRx, uint256 enRy, uint256 enCx, uint256 enCy)
        external view returns (bool);
}

```

Stub implementations (`StubMintVerifier` / `StubSpendVerifier`) ship for unit tests that exercise state transitions unrelated to the SNARK. The production `Adapter` contracts decode the proof as `(uint256[2], uint256[2][2], uint256[2])`, build the public-signals tuple, and forward to the snarkjs-generated Groth16 verifier. `mintVerifier` is a singleton `IMintVerifier` adapter that internally dispatches by `cms.length` to the correct pinned `MintBatchN*Groth16Verifier`; spend verifiers are likewise singletons. Swapping any of them is a single governance call.

8.4 Notes Contract

`src/Notes.sol` is constructed with `(buck, mintVerifier, spendVerifier, governance)`. The A-flavor spend verifier and the identity registry are wired post-construction via governance setters (`setSpendAVerifier`, `setIdentityRegistry`); until both are set, `spendACP` reverts with `Notes: A-spend disabled` or `Notes: identity registry not set`. Its state is intentionally lean – per-leaf insertion math now lives in the SNARK:

- **Verifier handles:** singleton `mintVerifier` (the adapter that dispatches by batch size), singletons `spendVerifier` for B-spend and `spendAVerifier` for A-spend, plus the `identityRegistry` used by `spendACP` for `verifySpendCP`. All swappable by governance.
- **Audit / double-spend state:** `mapping(uint256 => bool) nullifiers, uint256 noteFaceSum`.
- **Tree state:** `uint32 nextLeafIndex` and `uint256[ROOT_HISTORY_SIZE=30] roots` with `uint8 currentRootIndex`. `filledSubtrees`, `zeros[]`, and per-cm uniqueness checks are gone – duplicate openings are economically self-punishing via the deterministic nullifier (one spend nullifies every copy of the same opening; the duplicator forfeits their mint).

`mint(proof, oldRoot, newRoot, nextLeafIndex, totalFace, cms)` asserts `cms.length > 0`, checks `oldRoot == roots[currentRootIndex]` and `nextLeafIndex == self.nextLeafIndex` (stale-state guards), field-bounds each `cms[i]` against `FIELD_R`, dispatches to `mintVerifier` (which forwards to the per-N pinned Groth16 verifier internally), pulls `totalFace` BUCK via `transferFrom(caller, this, totalFace)`, writes `newRoot` into the ring buffer at `(currentRootIndex + 1) % ROOT_HISTORY_SIZE`, advances `nextLeafIndex` by `cms.length`, bumps `noteFaceSum` by `totalFace`, and emits `Minted(issuer,`

`totalFace`, `startIndex`, `count`, `newRoot`). Per-leaf `Appended` events are gone; the commitments live in calldata for off-chain replayers.

`spend(proof, root, nullifier, face, recipient)` (B-flavor) requires `recipient != 0`, `face > 0`, `isAcceptedRoot(root)` (root in the recent-roots window), and `!nullifiers[nullifier]`. Forwards to `spendVerifier`, marks the nullifier burned, decrements `noteFaceSum`, and calls `buck.transfer(recipient, face)`. Because the Notes pool is bound under the Carrying flag, `buck.transfer` dispatches to the carrying path automatically – the recipient absorbs the pool’s average demurrage age via `buckSeconds`, preserving the SNARK’s `face === v` invariant at the ERC-20 boundary. `Spent(nullifier, face, recipient)` closes the spend.

`spendACP(proof, root, nullifier, face, recipient, E_n, cpProof)` (A-flavor) is identical except it also calls `identityRegistry.verifySpendCP(msg.sender, recipient, E_n, cpProof)` atomically; both verifications must succeed.

9 End-to-End Walkthrough

Alice owns a \$500K Calgary home (land \$200K, structure \$300K). Two insurers (LandCo, HomeCo) have issued `BUCK_CREDIT` NFTs; HomeCo’s structure credit depreciates `LINEAR` at 200bp/yr against a 20 % floor. Three years on, and before Alice has minted anything:

Token	Insurer	Face Value	Depreciation	Depreciated Face	Premium Rate
#42	LandCo	200,000	NONE	200,000	50bp
#43	HomeCo	300,000	LINEAR, 200bp, 20% fl	285,600	200bp

Note what is *not* in that table. `activatedValue` is zero on both credits, because activation happens only inside `Buck.mint`; so `totalCurrentValue(alice) = 0` and Alice’s `creditLimit` is zero too. The credit limit is not "what she could borrow" – it is "what she has already drawn coverage for". Her `capacity` to mint is bounded per credit by `faceValue - mintsBacked`.

9.1 Step 0: Identity Registration (One-Time)

Alice obtains a PS-signed credential off-chain from a trusted issuer (the Alberta government, ATB Financial, etc.), blinds it into a presentation (A, B) , generates a fresh ElGamal key pair, encrypts her identity point $M = m \cdot G$ under her public key, and produces the Schnorr-family NIZK binding the presentation to the ciphertext and the key. `IdentityRegistry.register(issuer, pk, E, (A, B), proof)` verifies the proof via BN254 precompiles in 443,892 gas (measured). After this call `isVerified[alice] = true` and Alice can touch BUCK. She repeats steps 2-3 for every Ethereum account she controls; the issuer never sees those accounts.

9.2 Step 1: Mint

Alice calls `BUCK.mint(50_000e6)` – the no-arg form, so the built-in selector sorts her credits by `premiumRate` ascending: LandCo (50bp) first, then HomeCo (200bp). At `BUCK_K = 1e18`, and writing BUCK amounts rather than raw 6-decimal units:

1. `buckK.compute()` runs the PID cadence. The static controller returns `fundingFactor() = 0=`, so the counter-cyclical reserve gate is disabled and no pre-mint balance is captured.
2. The allocator walks #42 first: `denom = 10000 - 50*10 = 9500`, `avail = 200,000 - 0`, `netCap = avail * 9500 / 10000 = 190,000`. Since `netCap >= 50,000` the closed-form partial fill

lands it in one credit: $\text{take} = \text{ceil}(50,000 * 10000 / 9500) = 52,631.578948$, $\text{principal} = \text{take} - 50,000 = 2,631.578948$. #43 is never touched.

3. `activateFromBuck(#42, alice, take)` grows `activatedValue` to `take`, and `mintsBacked[#42]` with it. From this moment #42 cannot be transferred.
4. `insurancePool` is credited the principal and Alice's signed raw is debited the same: the principal's 10 % assumed ROI (263.16/yr) exactly funds the 50bp annual premium on 52,631.58 of coverage (263.16/yr), in perpetuity.

End state, all of it live-derived rather than stored:

```

activatedValue[#42] = mintsBacked[#42] = 52,631.578948
creditLimit(alice)      = 52,631.578948    (#42 is NONE-schedule, so no haircut)
signedRawBalanceOf(alice) = -2,631.578948
balanceOf(alice)        = 50,000.000000    (limit - used)
totalSupply             += 2,631.578948    (the pool's positive contribution)

```

Alice now holds no BUCK at all and can spend 50,000 of it. Had the selector reached #43 instead, she would have got the same 50,000 – the gross-up in `mint(N)` and `burn(N)` would simply have activated more of #43's face to carry it, and charged premium on the 285,600 appraisal rather than the 300,000 the structure was worth when it was new.

9.3 Step 2: Approve Bob (Identity Handshake)

Alice's wallet reads Bob's `pk_bob` from `IdentityRegistry`, decrypts `E_alice` (it knows `sk_alice`) to recover `M`, and re-encrypts under `pk_bob` with fresh randomness as `E_bob = (r_b · G, r_b · pk_bob + M)`. It produces a Chaum-Pedersen NIZK `pi_CP = (e, s_1, s_2)` proving `E_bob` encrypts the same `M` registered in `E_alice`, with Fiat-Shamir bound to `msg.sender = alice`, `spender = bob`, `chainid`.

`BUCK.approve(bob, 10_000e6, E_bob, pi_CP)` verifies both parties are `isVerified`, runs `IdentityRegistry`. (29K gas), stores `~receiptFragments[alice][bob] = keccak256(E_bob)`, calls `markApproved(bob)` to freeze his `isCarrying` flag, and writes the ERC-20 allowance.

Off-chain (e.g. via Holochain), Alice delivers the matching `identity_data` and `E_bob` ciphertext to Bob's wallet. Bob decrypts to recover `M'` and asserts `H(identity_data) * G === M'` – the two-channel binding from the Identity document.

9.4 Step 3: Bob Pulls the Transfer

`BUCK.transferFrom(alice, bob, 10_000e6)` debits the allowance, runs `_identityCheckedTransfer` (both parties verified, `~receiptFragments[alice][bob] != 0`), and dispatches to the Non-Carrying path (Alice is not Carrying). Tokens move; `BuckTransferReceipt(alice, bob, 10_000e6, fromHash, toHash)` is emitted. An auditor with subpoena power over either Alice or Bob can recover the underlying identities; an outside observer learns only that addresses Alice and Bob exchanged 10,000e6.

9.5 Step 4: Alice Denominates as Notes

Alice picks flavour A2 and values [100e6, 25e6] (totalFace 125e6). Her wallet pads the live pair with 14 dummies (`v=0`) so the batch lands on the smallest pinned circuit (`N=16`), snapshots (`oldRoot`, `nextLeafIndex`) from chain, derives the depth-20 sibling paths in submission

order, and runs `snarkjs groth16 fullProve` against `build/snark/mint_batch_n16/mint.wasm` and `mint_final.zkey`. The proof triple is abi-encoded into `proofBytes`.

On-chain:

1. `BUCK.approve(notes, 125e6, E_notes, pi_CP_notes)` – the always-CP rule applies even though `Notes` is bound under a Public Identity. The receipt fragment is openable on subpoena.
2. `Notes.mint(proofBytes, oldRoot, newRoot, nextLeafIndex, 125e6, [cm[0]..cm[15]])` – the adapter dispatches internally by `cms.length=16` to its pinned `MintBatch16` Groth16 verifier. On success, 125e6 BUCK moves from Alice to `Notes`, `roots[]` advances to the SNARK-attested `newRoot`, `nextLeafIndex` advances by 16, `noteFaceSum + 125e6=`. Alice keeps the live openings privately; the dummies are discarded.

9.6 Step 5: Spend a Note

Later, the holder of one of Alice’s notes (Alice herself, or a bearer she off-chain handed the opening to) redeems `cm[0]` for 100 BUCK to `recipient`:

1. The wallet reconstructs (`pathElements[20]`, `pathIndices[20]`) for `cm[0]` by replaying the `Minted` event log and pulling `cms[]` from each batch’s calldata.
2. `snarkjs groth16 fullProve` against `spend.wasm / spend_final.zkey` produces a proof binding [`currentRoot`, Poseidon-3(`rho`, `idHash`, 4242), 100e6, `recipient`, 1] to the witness.
3. `Notes.spend(proofBytes, currentRoot, nullifier, 100e6, recipient)` – the contract checks the root window, rejects a previously-seen nullifier, runs the adapter (~360K gas total), marks the nullifier burned, decrements `noteFaceSum`, and calls `buck.transfer(recipient, 100e6)`. `Notes` is bound Carrying, so the transfer dispatches to the carrying path automatically: the recipient receives the full 100e6 and absorbs an amount-weighted share of the pool’s average demurrage age via `buckSeconds`.

If HomeCo later reappraises the structure at \$280K, that is allowed – #43 carries no activated coverage, so the floor in Insurer Updates does not bind, and Alice’s future capacity against it drops accordingly. What HomeCo *cannot* do is write #42’s coverage down: LandCo is #42’s insurer, and even LandCo may not reappraise below the 52,631.58 Alice has bought.

Meanwhile #42 keeps depreciating on its schedule – `NONE`, so not at all here, but a depreciating credit would pull `creditLimit` down under Alice continuously. If it fell below her drawn 2,631.58 she would be underwater: she could not burn until she reacquired BUCK and repaid, her insurance would stay in force regardless, and the Jubilee relief accruing against #42’s coverage-age would shrink the cost of closing the position year on year.

10 Gas Cost Snapshot

Approximate gas costs from the current Foundry suite (Solidity 0.8.28, cancan, via-IR, optimizer at 200 runs). PID and oracle figures are estimates against unwritten oracle wiring.

Operation	Gas (approx)	At 30 gwei	Notes
BUCK_CREDIT.updateCredit()	~80K	~\$6	Multiple SSTOREs (in
IdentityRegistry.register()	~444K	~\$36	credential proof, 3 pair
IdentityRegistry.verifyApprove	~29K	~\$2	CP proof, per counterp
IdentityRegistry.verifySpendCP	~36K	~\$3	CP-DLEQ via EIP-196
BUCK.approve() (identity)	~97K	~\$8	29K CP + 46K allowa
BUCK.transfer / transferFrom	~80K	~\$6	Identity check + receip
– reentrancy guard, per guarded call	~600	–	TSTORE/TLOAD; no
– credit scan, sender holding 10 credits	~28K	~\$2	Live creditLimit; 6K w
– demurrage routed to a designated payer	~900	–	Warm payer slot; 0 if n
BUCK.mint() (static BUCK_K, 1 NFT)	~150K	~\$12	NFT walk + allocator
BUCK.mint() (static BUCK_K, 2-NFT spillover)	~210K	~\$17	+ second NFT crystal
BUCK.burn() (1 NFT)	~120K	~\$10	Mirror of mint
BUCK.mint() (with PID update, 4-pool basket)	~450K	~\$36	+ 4 V3 TWAP reads -
BUCK.mint() (PID cached, within dT)	~155K	~\$12	one SLOAD on the ca
Notes.mint (mint_batch, N=1)	~461K	~\$37	Groth16 verify (~216K
Notes.mint (mint_batch, N=16)	~624K	~\$50	~39K/leaf – shipped s
Notes.mint (mint_batch, N=32)	~1.12M	~\$90	~35K/leaf (verifier ~42
Notes.spend (Groth16, B-shape)	~360K	~\$29	Groth16 verify + Carr
Notes.spendACP (Groth16 + CP-DLEQ, A-shape)	~410K	~\$33	+ verifySpendCP (~36
BUCK_CREDIT.currentValue / BUCK_K.currentBuckK	0	Free	View

The PID update cost is amortised across all mints within a dT window (60 s default; only the first mint after each window pays the oracle-read + PID-math cost; every other mint sees the cached one-SLOAD path). `Buck.burn` calls `compute()` too, so burns share the amortisation. The same contracts deploy on EVM L2s (Arbitrum, Optimism, Base) where gas is 10-100x cheaper.

11 Security Considerations

11.1 Oracle Manipulation

BUCK/USDT and basket pool prices use Uniswap V3 TWAPs (600 s shipped, governance-tunable per pool), so manipulation requires sustained capital commitment over the full window proportional to pool depth. The `BuckKControllerV3TwapTest` suite verifies that a single-block flash-shape attack driving BUCK to \$0.50 leaves spot collapsed but TWAP unchanged at ~\$1.000002, with the PID computing off the unchanged TWAP. When/if Chainlink basket feeds are adopted later, the Chainlink branch of `_getBasketCost` must verify `updatedAt` is within an acceptable window (~1 hour) and revert if stale.

11.2 Unvetted Insurers – the open hole

This is the one thing on this page that must close before mainnet, and it is not closed.

`createCredit` is permissionless. Any address may issue a BUCK_CREDIT naming itself the insurer; there is no vetting of any kind. `updateCredit` is gated on `msg.sender == insurer`, but becoming an insurer in the first place is not gated at all.

The consequence is unbacked issuance, not merely a nuisance. A credit's `faceValue` and `premiumRate` are whatever its issuer says they are, and a zero-premium credit costs nothing to

draw on – `poolPrincipal == 0`, so the counter-cyclical funding gate is inapplicable by design. So a single verified account can write itself a policy on an asset it never names, and mint against it:

```
mallory issues herself a zero-premium credit, faceValue 1,000,000,000
mint(500,000,000)    -> principal paid    0
                        spendable          500,000,000 BUCK
                        transferred on     totalSupply 0 -> 500,000,000
```

Nothing downstream catches it. `totalCurrentValue` sums whatever credits an account holds without asking who issued them, and the identity layer only establishes that the *minter* is a registered person – not that the *insurer* is good for anything.

The recipient opt-in below does not close this, and cannot: the attacker is both insurer and client, so she accepts herself. Only vetting the insurer closes it. The intended shape is deliberately **not** a governance allowlist – issuance stays open to anyone who can demonstrate they are good for the claims: on-chain solvency (a buffer held, and provably available to the parametric payout trigger) plus a rating. That system is unbuilt. Until it exists, BUCK’s supply is bounded only by the honesty of everyone holding a registered identity.

11.3 Recipient Opt-In

A credit only lands where its recipient asked for it. A client calls `setCreditIssuer(insurer, true)` before that insurer may issue to them; the flag is per-pair, revocable, and governs only new issuance, since credits already held may be backing BUCK. The rule is uniform – naming yourself the insurer does not exempt you – which keeps the invariant statable in one line and declines to open a carve-out shaped exactly like the attack above.

What this closes is the gas-griefing vector. A credit costs its holder gas forever after: `totalCurrentValue` walks `tokenOfOwnerByIndex` over *every* token the account owns, and since the credit limit is read live, that walk runs on every outbound transfer from a non-Carrying account. Without the opt-in an attacker could raise a chosen address’s transfer cost without bound for the price of the mints, and the victim could not refuse delivery – ERC-721 `_mint` needs no consent from the receiver. A holder can still dispose of an unwanted credit by transferring it away, since an unactivated credit is freely transferable, but that is a remedy they should never have had to reach for.

It is not a substitute for vetting. It bounds who may put a credit in your hands; it says nothing about whether that credit is worth anything.

11.4 Insurer Trust

An insurer may reappraise `faceValue` downward, but not below coverage the holder has already bought (see Insurer Updates). So an insurer cannot cancel a policy, cannot strand a position, and cannot by itself put a holder into default: the worst it can do is reduce future borrowing capacity against that asset, and – via the depreciation schedule – let the holder’s credit limit decline over time. A holder pushed below their limit that way is in the supported underwater state, not a defaulted one.

That leaves the residual risks from an insurer who *has* been vetted: refusing to honour a claim, and setting a punitive `premiumRate` on new coverage. Multi-insurer market discipline is the natural check, and a rating that responds to claim behaviour is the intended mechanism. Time-locks on insurer updates (e.g. 7-day delay for reductions > 20 %) remain a future enhancement. For the prior question – how an insurer becomes vetted at all – see Unvetted Insurers.

11.5 Reentrancy

Every state-mutating entry point on **Buck** carries a transient-storage mutex; the design and the measured cost are in Reentrancy Guard. The guard has no exemptions.

The guard is defence in depth rather than a fix for a live hole. Every outbound call **Buck** makes today is either a **STATICCALL** – including the PID controller’s oracle reads, which sit behind an **internal view _readReferences()** – or a call into one of the three immutable, address-locked system contracts, none of which reach arbitrary code. What the guard protects against is the future: **BuckCredit** behind a UUPS proxy (see Future Directions), or any ERC-721 receiver hook added to the activation path. The window it closes is real and specific: **_allocateMint** activates coverage – which grows credit headroom immediately – before debiting the minter the pool principal, so mid-mint **balanceOf** overstates, and **_subBalance** does not re-check the limit on the way out.

Note that view functions are not guarded and cannot be: read-only reentrancy against **balanceOf** is a hazard for *integrations*, which must not sample a BUCK balance from inside a callback they do not control. See Integrating with BUCK.

11.6 Pool Solvency

insurancePool holds the cumulative pool principal for every outstanding mint. Burn requires the pool’s spendable balance to cover the proportional refund; if the pool is drained out-of-band the burn reverts. Today no path drains the pool: it is not a verified identity (so no identity-checked transfer can move BUCK out of it) and holds no NFTs (so it cannot self-burn via the allocator). Adding governance-controlled disbursement requires explicit accounting of how the pool’s invariant (**poolBalance** >= sum of all outstanding mint principals) is preserved.

12 Future Directions

This implementation prioritises clarity and correctness; alternatives the architecture document flags for later study:

- **ERC-1155 batch credits** if standardised credit *classes* emerge (e.g. "Calgary residential, HomeCo, LINEAR 200bp"), enabling **safeBatchTransferFrom** for portfolio management and reduced gas for accounts holding many same-class credits. Per-token depreciation start dates partially negate the fungibility benefit, so applicability depends on the secondary market.
- **ERC-3643 (T-REX)** if regulatory KYC compliance becomes a deployment requirement. Native ONCHAINID, programmable transfer restrictions, agent-role granularity, and lost-key recovery via identity verification. This is the likely production standard if Alberta deploys BUCK under provincial regulation.
- **ERC-4626 InsurancePool** – a vault-shaped wrapping of the existing **insurancePool** address, so pool deposits become standardised mutual-insurance shares with **deposit** / **shares** / yield composability. Mechanical layer on top of the current single-address pool.
- **L2 deployment.** The contracts are EVM-compatible and deploy on Arbitrum, Optimism, Base, or Polygon with no changes. Cross-chain BUCK via canonical bridges or LayerZero/Axelar would enable multi-chain liquidity while keeping a single BUCK_K oracle on the settlement layer.

- **Coverage-ratio premium.** The current per-NFT `premiumRate` is flat. A loss-layer-style effective-rate derivation – $\text{rate}(V, A, D) = \text{lambda} * \text{coverage} * (1 - D/V)^2 / 2$ under uniform loss – would price closer-to-full-coverage activations more aggressively, which matches actuarial intuition for parametric insurance with deductibles. The allocator’s sort key would move from raw `premiumRate` to a midpoint-evaluated effective rate; everything else carries through.
- **Recursive SNARKs above N=512.** Mint batches above N=64 hit the EIP-170 24 KB bytecode ceiling on stock Groth16 verifiers. Calldata-IC delivery (hash-pinned) buys headroom to N=128 or 256; Nova / Halo2 / Plonky3 recursion is the natural fix above N=512.
- **Buck as a Diamond (EIP-2535).** Buck today is a single monolith covering identity, demurrage, NFT-credit accounting, ERC-20 wire-protocol, BuckBasket direct-mint hooks, and Jubilee bookkeeping. Splitting these into facets (identity facet, demurrage facet, mint-burn facet, ERC-20 facet) is the natural modularization path: facets share Buck’s storage layout (`AccountState`, `_totalSupply`, `mintsBacked`), function selectors don’t collide within a single ERC standard, and per-facet upgradability lets bug fixes ship without redeploying the whole stack. Cross-contract calls into `BuckCredit` and `BuckKController` stay external – the ERC-20 / ERC-721 selector collision detailed in BUCK_CREDIT forces `BuckCredit` to remain a sibling.
- **Independently upgradeable BuckCredit (UUPS or transparent proxy).** Keep Buck’s immutable `buckCredit` address stable while patching `BuckCredit`’s logic in place. Combined with the Diamond split above, this lets the system evolve facet-by-facet and contract-by-contract without the Big Bang redeploy that an immutable monolith requires.

13 Status

Current Foundry suite: 53 test files cover all six contracts in tree (`BuckCredit`, `IdentityRegistry`, `Buck`, `BuckBasket`, `BuckKControllerStatic` + `BuckKControllerDirect` + `BuckKController`, `Notes`) plus dedicated suites for the NFT-credit machinery (`BuckCreditLimit.t.sol`, `BuckCreditBacking.t.sol`, `BuckCreditReappraisal.t.sol`, `BuckSignedBalance.t.sol`), the reentrancy guard and delegated demurrage (`BuckReentrancy.t.sol`, `BuckDelegatedDemurrage.t.sol`, `GuardBench.t.sol`), the SNARK toolchain (`mint_batch.circom` at pinned $N \in \{1, 2, 4, 8, 16, 32\}$, `spend.circom`, `spend_a.circom` V2), and a Python reference wallet (`alberta_buck.wallet`) that emits canonical test vectors the Solidity tests load via `vm.readFile`. One `BuckKControllerForkTest` requires a live mainnet RPC and is skipped offline.

The BUCK_K and BuckBasket test suites cover:

- `BuckKControllerBaseTest` – abstract-base harness exercising the shared PID mechanics (priming, `dS` compensation, `dT`, `dTMax`, anti-windup, integral convergence, `fundingFactor`) against a synthetic `_readReferences` override.
- `BuckKControllerStaticTest` – governance-set constant, `setBuckK` guards, `fundingFactor` always-UNIT.
- `BuckKControllerDirectTest` – the USD-free direct embodiment wired to a mock basket source; `reprime` access control, dilution-discontinuity absorption, sign-convention canaries on the `basketValue > 1.0` and `< 1.0` branches.

- **BuckKControllerUnitTest** – harness with mocked Chainlink feeds, mocked BUCK price. Includes priming, dS compensation, dTMax clamp, anti-windup, and multi-step integral convergence.
- **BuckKControllerV3Test** – locally deployed Uniswap V3 factory, all four basket pools at reference prices, BUCK/USDT pool with a 1e17 full-range LP (~100 K BUCK / 100 K USDT, matching the design narrative). Spot-price reads, swap-driven price drift, and PID response.
- **BuckKControllerV3TwapTest** – same V3 setup with `twapInterval = 600 s` on every pool, plus a warmup loop that walks forward writing observations on each pool. Verifies the TWAP path smooths a flash-shape spike (spot \$0.50, TWAP \$1.000002) and tracks sustained drift over the 600-s window (TWAP converges from \$1.00 → \$0.97 → \$0.95 over a full window at \$0.95 spot).
- **BuckBasketTest** – direct-mint deposit/redeem flows, `addBasketToken` renormalization, proportional-allocation correctness across overweight/underweight pools, "sell high / buy low" redemption invariants, shortfall-cover greedy swap, and the `reprime` callback into `BuckKControllerDirect`.
- **BuckEquilibriumScenarioTest** – multi-month closed-loop scenario; ticks `BuckBasket.depositToken` / `redeem` and `Buck.mint` / `burn` across simulated price drift to exercise `fundingFactor` and the buckK response in concert.
- **BuckKArbScenarioTest** – driven-arbitrageur scenario (`BuckKArbScenarioTest`, 2-year horizon) that exercises the External-Basket PID against an Alice agent who trades around the PID equilibrium; emits `arb-scenario.json` for the Python visualizer.

The Python simulation harness in `alberta_buck/sim/` externally drives an anvil instance with the deployed contracts (see `make sim*` targets and the project memo on the externally-driven sim) to replay multi-agent scenarios – `direct_mint.py` and `scenario.py` for the basket-side flows, `agents.py` for arb / depositor / redeemer agents, `rebalancer.py` for the cross-pool recycling sanity check, `plot_basket_model.py` / `plot_rebalancing.py` / `plot_routing.py` for visualization.

Outstanding work tracked in companion memos:

- **Equilibrium-seeking dynamic-issuance test** – closed-loop integration of `BuckBasket`, `BuckKControllerDirect`, `BuckCredit`, and `Buck` against an external market driver and an arb agent that responds to buckK changes by minting/burning `BuckCredit` backed BUCK. Should emit a snapshot tape suitable for the existing `plot_*` visualizers and double as the regression for Dynamic Issuance: Why BuckBasket Needs BUCK_K. Highest-priority validation for the closed-loop control claim.
- **Resolve BUG #5..#11/#13 in BuckBasket** – per-withdrawal slippage guards, TWAP-bounded swap limits, TWAP-driven overweight/underweight reads, proportional reinvestment, symmetric TOKEN-deposit routing, basket-mint accounting in `fundingFactor`, treasury-reinvest event, graceful tail-redemption.
- **Mainnet-fork BUCK_K test** – exercise `BuckKController` against the four real on-chain pools at a pinned recent block, verifying basket cost reads sensibly against live data. Cheap validation step before mainnet.
- **Multi-block historical replay** – a backtest harness that walks `BuckKController` through 12 months of EVM history at a chosen block cadence, for tuning `Kp/Ki/Kd` against a real price tape. Highest-value validation; biggest engineering lift.

- **Serialised B1 sub-notes** (`spend_serialized.circom` + per-parent bitmap) – design validated end-to-end by `scripts/snark/serialized_notes_poc.py` at `N_fam` in `{256, 1024, 4096}`; circuit port and contract bitmap support are the implementation gap. See alberta-buck-notes-serialized.org.
- **Production trusted-setup ceremony** – replace dev entropy in `scripts/snark/setup.sh` with a multi-party ceremony per pinned circuit before mainnet.

The cryptographic surface is described by an executable Python reference; CI regenerates the canonical JSON test vectors and `git diff --exit-code`'s them against committed copies, so any drift between the spec, the wallet, and the Solidity tells us immediately which of the three is wrong. No `vm.ffi` shells out to Python at test time – the wallet emits vectors once, the Solidity reads them many times.